

哈爾濱工業大學（深圳）
HARBIN INSTITUTE OF TECHNOLOGY

实验设计报告

开课学期：2025 年春季

课程名称：计算机系统

实验名称：Lab2 TinyShell

实验性质：课内实验

实验时间：5/14 地点：T2617

学生班级：1 班

学生学号：220110120

学生姓名：文梓锋

评阅教师：

报告成绩：

实验与创新实践教育中心印制

2025 年 4 月

1. 需求分析

（列出 TinyShell 应支持的基本功能（如执行外部命令、内置命令、命令行解析等））

编写的 TinyShell 将具备以下核心功能：

1. 命令行解析：
   (1) 能够将输入的命令字符串正确解析为可执行命令及其参数。
   (2) 支持识别后台任务（如命令魔为&）、单引号包裹的参数等等
2. 内置命令支持：实现常用的内置命令，如 quit、jobs、bg、fg 等。
   (1) quit：退出 shell
   (2) jobs：列出任务
   (3) bg：后台运行
   (4) fg：前台运行
3. 进程创建与管理：
   (1) 使用 fork() 和 exec() 系列函数创建子进程，并管理其执行流程。
   (2) 管理 job 列表：包括作业的添加、删除、状态更新等等
   (3) 支持前台（FG）、后台（BG）、停止（ST）作业状态
4. 信号处理机制：
   (1) 能够处理操作系统或子进程发送的信号，如 SIGINT、SIGTSTP、SIGCHLD 等。
   (2) 能够向前台作业发送信号、实现终止或暂停前台作业
   (3) 通过信号处理函数保证作业状态的正确维护、防止竞态条件

2. 设计思路

（设计 TinyShell 的整体架构，包括主函数、命令行解析函数、内置命令处理函数等。确定函数之间的调用关系和参数传递方式。）

1. 主函数 (main)
- 初始化信号处理程序 (Signal)
- 初始化作业列表 (initjobs)
- 进入 Shell 的主循环：
  - 打印命令提示符
  - 读取命令行输入 (fgets)
  - 调用 eval 评估命令

2. 评估命令 (eval)
- 解析命令行参数 (parseline)
- 判断是否是内建命令 (builtin_cmd)：

  1. 是：直接执行命令
  2. 否：创建子进程 (fork)

  (1) 子进程：

  ① 取消信号阻塞

  ② 执行命令 (execve)

  (2) 父进程：根据前台/后台状态：

  ① 前台任务调用 waitfg 等待执行完毕

  ② 后台任务打印进程信息

3. 内建命令 (builtin_cmd)
- quit: 退出程序
- jobs: 显示作业列表 (listjobs)
- bg / fg: 后台/前台任务转换 (do_bgfg)

4. 任务控制 (do_bgfg)
- 解析 %jobid 或 pid
- 发送 SIGCONT 恢复暂停的任务
- 更新任务状态为 BG 或 FG
- 如果是 FG，调用 waitfg 等待任务结束

5. 前台任务等待 (waitfg)
- 阻塞当前进程，直到前台任务被信号处理程序 (sigchld_handler) 回收。

6. 信号处理程序
- sigchld_handler: 回收子进程，处理僵尸进程
- sigint_handler: 处理 Ctrl+C，终止前台进程
- sigtstp_handler: 处理 Ctrl+Z，暂停前台进程

函数之间的调用关系和参数传递方式

主循环执行流程

1. main
   (1) 进入主循环，等待用户输入命令行
   (2) 调用 eval(cmdline) 处理输入的命令
2. eval(cmdline)
   (1) 主要负责解释命令并执行
   (2) 调用 parseline(cmdline, argv) 解析命令行字符串为参数数组 argv
   (3) 判断是否是内建命令
      ① 内建命令调用 builtin_cmd(argv)
         1) quit: 退出程序
         2) jobs: 调用 listjobs(jobs) 列出作业
         3) bg 或 fg: 调用 do_bgfg(argv) 执行后台或前台任务切换
   (4) 如果不是内建命令，则通过 fork 创建子进程，并调用 execve 执行命令
   (5) 如果是前台任务，调用 waitfg(pid) 等待前台任务结束

内建命令执行流程

1. builtin_cmd(argv)
   (1) 判断是否是内建命令：quit, jobs, bg, fg
   (2) 对于 bg 或 fg 调用 do_bgfg(argv)
2. do_bgfg(argv)
   (1) 解析参数是 %jid 还是 pid
   (2) 如果是 bg，改变进程状态为 BG，并发送 SIGCONT 信号恢复执行
   (3) 如果是 fg，改变进程状态为 FG，并调用 waitfg(pid) 等待任务完成

等待前台进程结束

1. waitfg(pid)
   (1) 使用 sigsuspend 暂停当前进程，等待 SIGCHLD 信号唤醒
   (2) 被唤醒后判断是否是前台任务被回收，若是则继续执行

信号处理流程

1. sigchld_handler(int sig)
   (1) 响应 SIGCHLD 信号
   (2) 通过 waitpid 处理所有子进程的状态变化
   (3) 进程终止（正常退出或被信号杀死）则调用 deletejob 从作业表中删除
   (4) 进程暂停（如 SIGTSTP），则更改状态为 ST
   (5) 进程恢复（如 SIGCONT），状态不变
2. sigint_handler(int sig)
   (1) 响应 SIGINT (Ctrl + C)

(2) 向前台进程组发送 SIGINT 信号终止它们

3. sigtstp_handler(int sig)

(1) 响应 SIGTSTP (Ctrl + Z)

(2) 向前台进程组发送 SIGTSTP 信号暂停它们

调用关系如下图所示

```
main
└── eval(cmdline)
    ├── parseline(cmdline, argv)
    ├── builtin_cmd(argv)
    │   ├── listjobs(jobs) [jobs 命令]
    │   └── do_bgfg(argv)  [bg/fg 命令]
    ├── fork()
    │   └── execve(argv[0], argv, environ) [子进程执行命令]
    ├── addjob(jobs, pid, state, cmdline)
    └── waitfg(pid) [等待前台任务结束]
        └── sigsuspend(&prev_mask) [等待信号处理]
```

3. 核心函数实现

（深入分析核心函数的实现）

3.1 void eval(char *cmdline)函数

函数功能：

- eval 函数用于解析并执行用户输入的命令行。如果是内建命令（如 quit、jobs、bg 或 fg），则直接在当前进程中执行。如果是外部命令，则创建子进程并在子进程中执行，同时根据是否是后台任务决定是否等待子进程结束。

参　　数：

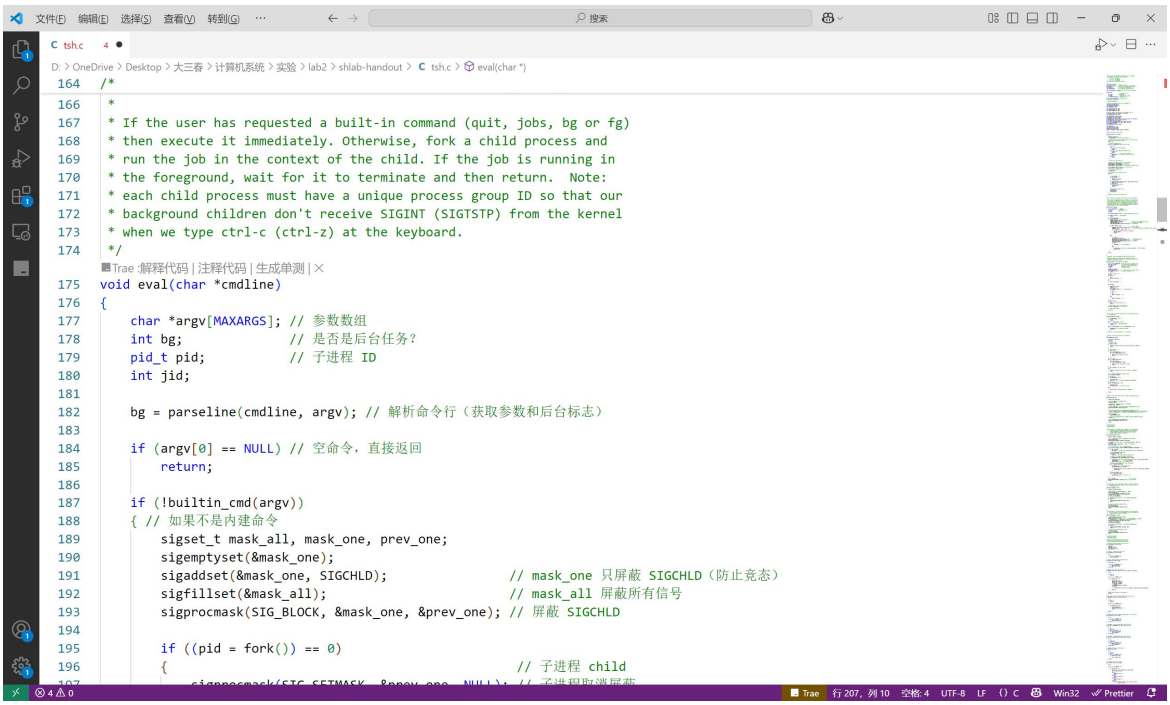
- cmdline: 用户输入的命令行字符串。

处理流程：

1. 解析命令行：使用 parseline 函数解析命令行，提取命令和参数到 argv 数组，同时判断是否是后台任务并设置 bg 标志位。
2. 空命令检查：如果 argv[0] == NULL，表示输入是空的，直接返回。
3. 内建命令检查：调用 builtin_cmd 函数检查是否是内建命令，如果是，则直接执行并返回。
4. 外部命令处理
   (1) 初始化信号屏蔽集：
      ① mask_one: 只屏蔽 SIGCHLD，防止子进程尚未添加到作业列表就被回收，避免出现竞争条件。
      ② mask_all: 屏蔽所有信号，用于保护临界区。
   (2) 使用 sigprocmask 设置信号屏蔽。
5. 创建子进程
   (1) 调用 fork 创建子进程：
      ① 子进程：
         1) 恢复信号屏蔽。
         2) 调用 setpgid(0, 0) 创建新的进程组，避免接收父进程的控制信号。
         3) 使用 execve 执行命令，若执行失败，打印错误信息并退出。
      ② 父进程：
         1) 如果是前台任务，调用 waitfg 等待子进程完成。
         2) 如果是后台任务，打印后台任务信息，包括作业 ID 和进程 ID。
6. 恢复信号屏蔽：临界区结束后，恢复原有的信号屏蔽状态。

要点分析：

1. 信号屏蔽：
   (1) 在 fork 之前屏蔽 SIGCHLD，防止竞争条件。
   (2) 在添加到作业表后恢复信号，保证作业不会在未注册之前被收割。
2. 进程组的创建：使用 setpgid(0, 0) 将子进程放到新的进程组，避免收到父进程的信号干扰（例如 Ctrl-C 和 Ctrl-Z）。
3. 后台任务的显示：如果是后台任务，主进程不会等待其结束，而是直接返回 shell 提示符。
4. 前台任务的等待：如果是前台任务，主进程会调用 waitfg，同步等待子进程结束。
5. 错误处理：execve 执行失败会打印错误消息，避免静默失败。



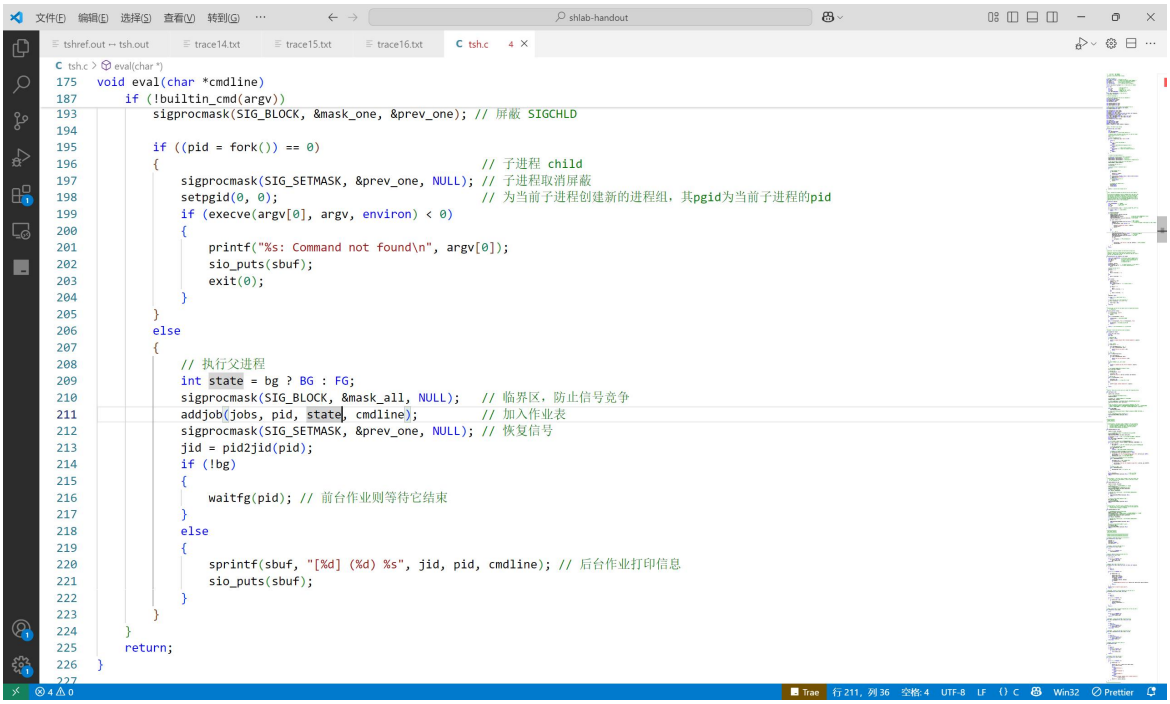
```c
/*
 *
 * If the user has requested a built-in command (quit, jobs, bg or fg)
 * then execute it immediately. Otherwise, fork a child process and
 * run the job in the context of the child. If the job is running in
 * the foreground, wait for it to terminate and then return.  Note:
 * each child process must have a unique process group ID so that our
 * background children don't receive SIGINT (SIGTSTP) from the kernel
 * when we type ctrl-c (ctrl-z) at the keyboard.
 */
Trae :解释代码 | 注释代码 | 生成单测 | ×
void eval(char *cmdline)
{
    char *argv[MAXARGS]; // 参数数组
    int bg;              // 是否是后台任务?
    pid_t pid;           // 子进程 ID
    int jid;

    bg = parseline(cmdline, argv); // 解析命令行（获取参数和后台标志）

    if (argv[0] == NULL) // 空命令，直接返回
        return;

    if (!builtin_cmd(argv))
    { // 如果不是内建命令
        sigset_t mask_all, mask_one, prev_one;
        sigemptyset(&mask_one);
        sigaddset(&mask_one, SIGCHLD);                // mask_one 只屏蔽 SIGCHLD（防止竞态）
        sigfillset(&mask_all);                        // mask_all 屏蔽所有信号
        sigprocmask(SIG_BLOCK, &mask_one, &prev_one); // 屏蔽 SIGCHLD

        if ((pid = fork()) == 0)
        {                                             // 子进程 child
            sigprocmask(SIG_SETMASK, &prev_one, NULL);
```



```c
void eval(char *cmdline)
    if (!builtin_cmd(argv))
        sigprocmask(SIG_BLOCK, &mask_one, &prev_one); // 屏蔽 SIGCHLD

        if ((pid = fork()) == 0)
        {                                             // 子进程 child
            sigprocmask(SIG_SETMASK, &prev_one, NULL); // 子进程取消屏蔽
            setpgid(0, 0);                            // 为当前子进程创建新的进程组，其pgid为当前子进程的pid
            if (execve(argv[0], argv, environ) < 0)
            {
                printf("%s: Command not found\n", argv[0]);
                sio_puts(sbuf);
                exit(0);
            }
        }
        else
        {
            // 执行父进程
            int state = bg ? BG : FG;
            sigprocmask(SIG_BLOCK, &mask_all, NULL);   // 临界区，防止信号竞争
            addjob(jobs, pid, state, cmdline);         // 加入作业表
            sigprocmask(SIG_SETMASK, &prev_one, NULL); // 恢复信号
            jid = pid2jid(pid);
            if (!bg)
            {
                waitfg(pid); // 前台作业则等待它结束
            }
            else
            {
                sprintf(sbuf, "[%d] (%d) %s", jid, pid, cmdline); // 后台作业打印信息
                sio_puts(sbuf);
            }
        }
    }
    return;
}
```

3.2 int builtin_cmd(char **argv)函数

函数功能：

- builtin_cmd 函数用于判断并执行用户输入的内建命令（Built-in Commands）。如果是内建命令，则直接在当前进程中执行并返回 1，表示已处理。如果不是内建命令，则返回 0，表示需要后续的 fork 和 exec 处理。

参　　数：

- argv: 指向命令和参数的字符串数组，argv[0] 是命令名，后续是命令的参数。

处理流程：

1. 检查是否是 quit 命令：使用 strcmp 比较 argv[0] 是否为 "quit"：如果匹配，则调用 exit(0) 退出程序。
2. 检查是否是 jobs 命令：如果 argv[0] 是 "jobs"：
   (1) 调用 listjobs(jobs) 列出所有作业信息。
   (2) 返回 1，表示是内建命令，已经执行完毕。
3. 检查是否是 bg 或 fg 命令：如果 argv[0] 是 "bg" 或 "fg"：
   (1) 调用 do_bgfg(argv) 处理后台或前台任务的切换。
   (2) 返回 1，表示命令已经执行。
4. 默认情况：如果以上都不匹配，返回 0，表示不是内建命令。

要点分析：

1. 内建命令的执行不需要创建子进程：这些命令都直接在父进程（Shell 进程）中执行，避免了 fork 的开销。
2. quit 命令直接退出整个 Shell：这是一个快速退出方式，直接调用 exit(0) 终止当前进程。
3. jobs 命令打印作业信息：调用了 listjobs(jobs) 函数来列出后台和前台作业信息。
4. bg 和 fg 处理：
   (1) 调用 do_bgfg(argv) 来实现作业的前后台切换：
      ① bg：将暂停的后台作业继续执行。
      ② fg：将后台作业拉回到前台执行。
5. 返回值的意义
   (1) 1 表示是内建命令且执行完成。
   (2) 0 表示不是内建命令，需要进入外部程序的执行流程。

```c
/*
 * builtin_cmd - If the user has typed a built-in command then execute
 *    it immediately.
 */
```

Trae :解释代码 | 注释代码 | 生成单测 | ×

```c
int builtin_cmd(char **argv)
{
    if (!strcmp(argv[0], "quit"))
    { // quit: 退出程序
        exit(0);
    }
    else if (!strcmp(argv[0], "jobs"))
    {
        listjobs(jobs); // jobs: 打印作业列表
        return 1;
    }
    else if (!strcmp(argv[0], "bg") || !strcmp(argv[0], "fg"))
    {
        do_bgfg(argv); // 调用处理 bg/fg 的函数
        return 1;
    }

    return 0; // 不是内建命令，返回 0 继续走 fork/exec
}
```

3.3 void do_bgfg(char **argv) 函数

函数功能：

- do_bgfg 函数用于执行内建命令 bg 和 fg，将指定的作业从后台恢复执行，或者将后台作业切换到前台执行。

参　　数：

- argv: 指向命令和参数的字符串数组。
  - argv[0]: 为命令名（bg 或 fg）。
  - argv[1]: 为进程 ID (PID) 或作业 ID (%jobid)。

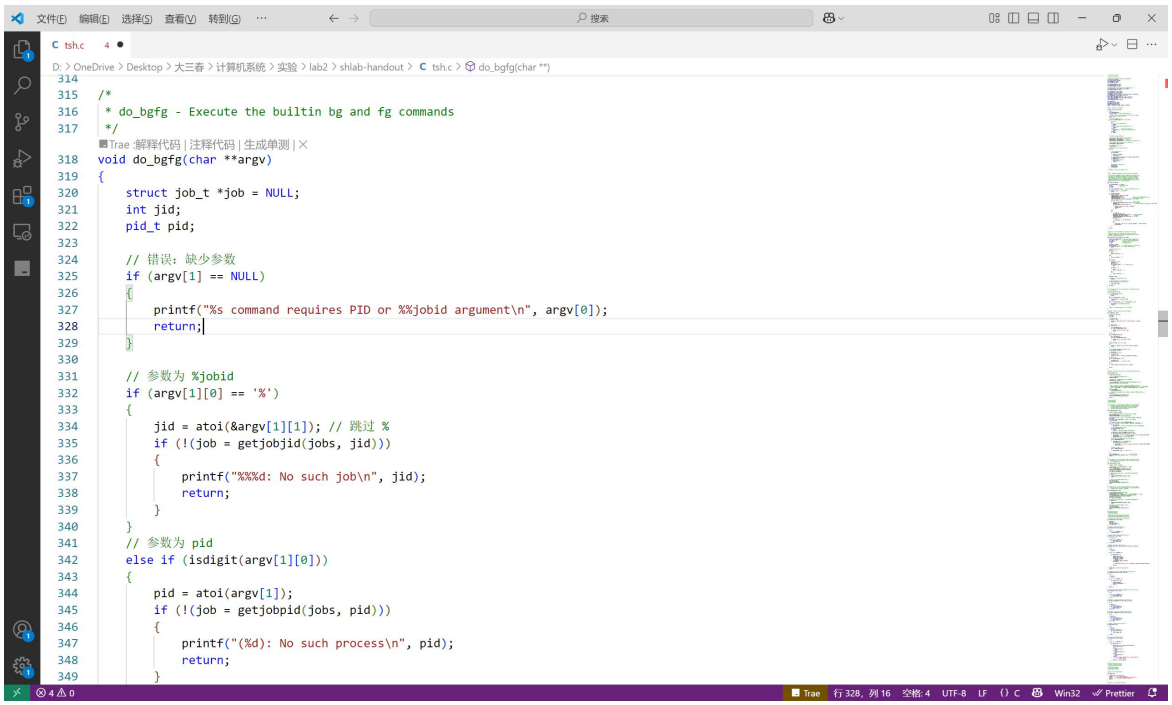
处理流程：

1. 检查参数是否缺失
   (1) 如果 argv[1] 为空，打印错误信息：<command> command requires PID or %jobid argument
   (2) 直接返回，不再执行后续步骤。
2. 判断是 %jobid 还是 pid
   (1) %jobid 的情况：
   ① 如果 argv[1] 以 % 开头，表示是作业 ID。
   ② 使用 atoi 转换 % 后面的部分为整数 jid。
   ③ 调用 getjobjid 查找对应的作业：
   ④ 如果找不到对应的作业，打印错误信息：%<jid>: No such job
   ⑤ 直接返回。
   (2) pid 的情况：
   ① 如果是数字，表示是进程 ID。
   ② 使用 atoi 转换为整数 pid。
   ③ 调用 getjobpid 查找对应的作业：
   ④ 如果找不到对应的作业，打印错误信息：(<pid>): No such process
   ⑤ 直接返回。
   (3) 错误参数：
   ① 如果既不是 %jobid 也不是 pid，打印错误信息：<command>: argument must be a PID or %jobid
   ② 直接返回。
3. 发送 SIGCONT 信号
   (1) 使用 kill(-(job->pid), SIGCONT); 发送 SIGCONT 信号：
   ① -(job->pid) 表示发给整个进程组，而不仅是单个进程。
   ② 这会唤醒暂停的作业继续执行。
4. 处理 bg 和 fg 的执行逻辑
   (1) 如果是 bg 命令：
   ① 设置作业状态为 BG。
   ② 打印后台执行信息：[jid] (pid) cmdline。
   (2) 如果是 fg 命令：
   ① 设置作业状态为 FG。
   ② 调用 waitfg(job->pid)，阻塞主进程，直到该作业结束。

5. 错误命令处理：如果既不是 bg 也不是 fg，打印：do_bgfg: unknown command <command>

要点分析：

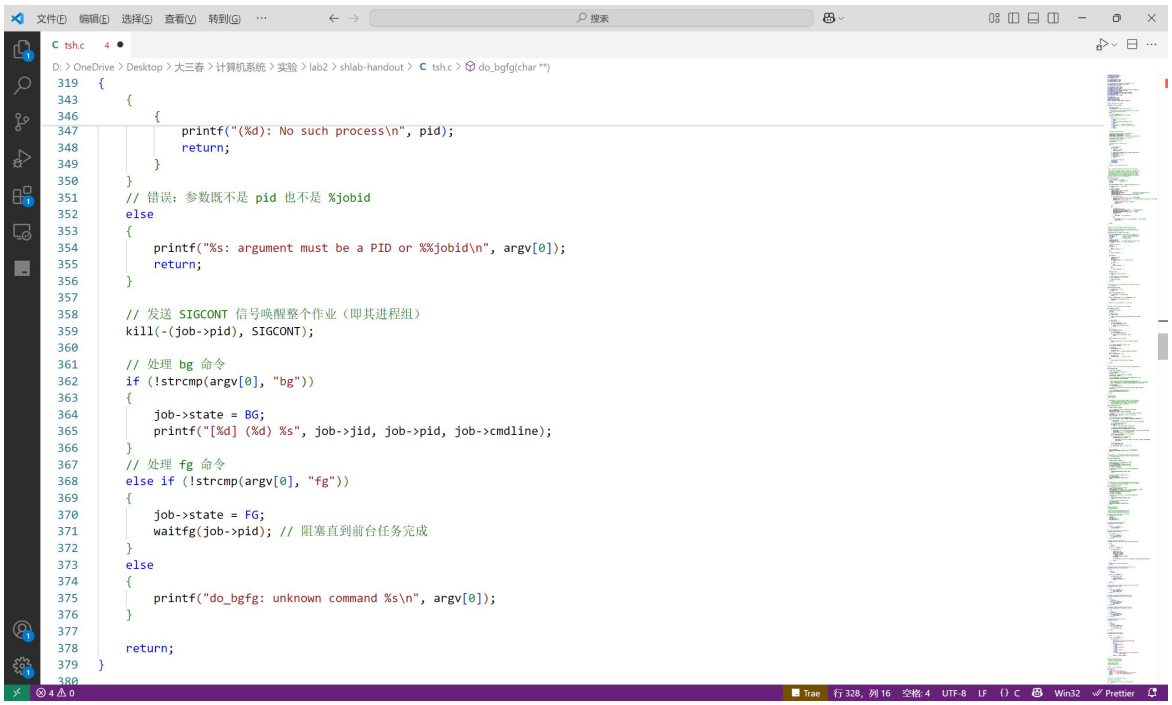
1. 区分作业 ID 和进程 ID
   (1) % 开头的是作业 ID (jid)，直接是数字的是进程 ID (pid)。
   (2) 使用不同的查找函数来确认作业是否存在。
2. 信号的发送
   (1) SIGCONT 信号用于唤醒暂停的进程，这可以是 Ctrl-Z 暂停后的恢复。
   (2) 使用负的 PID 表示发送给整个进程组，以确保所有相关进程同步恢复。
3. 后台与前台的区分
   (1) bg：后台执行，Shell 立即返回提示符。
   (2) fg：前台执行，Shell 会等待该任务完成再继续接受输入。
4. 阻塞等待
   (1) waitfg(job->pid) 会阻塞主进程直到前台任务结束，避免 Shell 继续接受命令。



```c
/*
 * do_bgfg - Execute the builtin bg and fg commands
 */
void do_bgfg(char **argv)
{
    struct job_t *job = NULL;
    int jid;
    pid_t pid;

    // 错误：缺少参数
    if (argv[1] == NULL)
    {
        printf("%s command requires PID or %%jobid argument\n", argv[0]);
        return;
    }

    // 参数为 %jobid
    if (argv[1][0] == '%')
    {
        jid = atoi(&argv[1][1]); // 跳过 %
        if (!(job = getjobjid(jobs, jid)))
        {
            printf("%%%d: No such job\n", jid);
            return;
        }
    }
    // 参数为 pid
    else if (isdigit(argv[1][0]))
    {
        pid = atoi(argv[1]);
        if (!(job = getjobpid(jobs, pid)))
        {
            printf("(%d): No such process\n", pid);
            return;
        }
```



```
{
    {
        {
            printf("(%d): No such process\n", pid);
            return;
        }
    }
    // 错误：参数既不是 pid 也不是 %jobid
    else
    {
        printf("%s: argument must be a PID or %%jobid\n", argv[0]);
        return;
    }

    // 发送 SIGCONT 信号唤醒整个作业（即其进程组）
    kill(-(job->pid), SIGCONT);

    // 处理 bg 命令
    if (!strcmp(argv[0], "bg"))
    {
        job->state = BG;
        printf("[%d] (%d) %s", job->jid, job->pid, job->cmdline);
    }
    // 处理 fg 命令
    else if (!strcmp(argv[0], "fg"))
    {
        job->state = FG;
        waitfg(job->pid); // 阻塞直到前台任务完成
    }
    else
    {
        printf("do_bgfg: unknown command %s\n", argv[0]);
    }

    return;
}
```

3.4 void waitfg(pid_t pid) 函数

函数功能：

- waitfg 函数用于阻塞父进程（Shell），直到指定的前台进程 pid 完成执行。期间父进程不会继续处理新的命令输入，保证前台任务的同步执行。

参　　数：

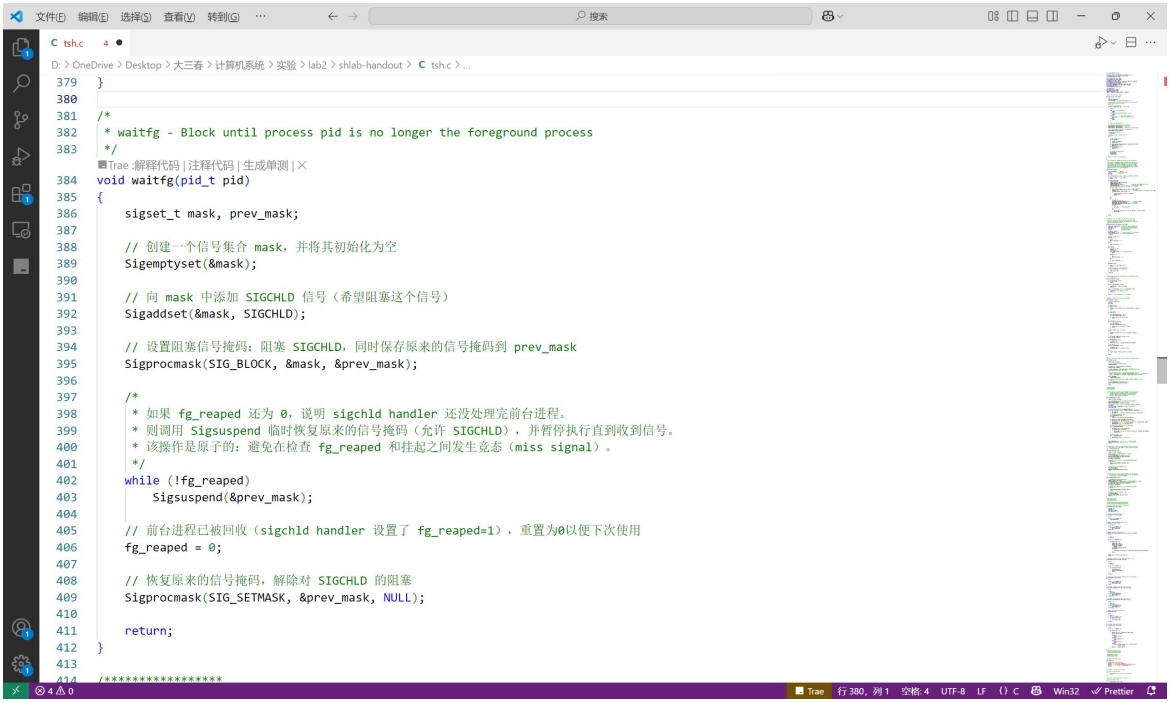
- pid: 前台任务对应的进程 ID。

处理流程：

1. 信号集初始化
   (1) 调用 Sigemptyset(&mask); 初始化一个空的信号集合。
   (2) 使用 Sigaddset(&mask, SIGCHLD); 将 SIGCHLD 信号添加到集合中：
      ① SIGCHLD 是子进程终止或停止时会发送给父进程的信号。
2. 阻塞 SIGCHLD 信号
   (1) Sigprocmask(SIG_BLOCK, &mask, &prev_mask);：
      ① 阻塞 SIGCHLD 信号，防止在临界区内被中断。
      ② 同时将之前的信号屏蔽状态保存到 prev_mask，便于恢复。
3. 等待前台任务结束
   (1) 使用 while (!fg_reaped) Sigsuspend(&prev_mask); 进入阻塞等待：
      ① fg_reaped 是一个全局变量，由 SIGCHLD 的信号处理程序修改。
      ② 如果子进程没有结束，则调用 Sigsuspend(&prev_mask);：Sigsuspend 原子性地恢复原信号屏蔽状态，并挂起等待信号到来。当收到信号（SIGCHLD）时恢复执行。
4. 子进程结束后的处理
   (1) 当 SIGCHLD 处理完毕，fg_reaped 会被设置为 1，while 循环结束。
   (2) 将 fg_reaped 重新置为 0，为下一次前台任务准备。
5. 恢复信号掩码
   (1) Sigprocmask(SIG_SETMASK, &prev_mask, NULL); 恢复之前的信号屏蔽状态，解除对 SIGCHLD 的阻塞。

要点分析：

1. 信号屏蔽的重要性
   (1) 在临界区屏蔽 SIGCHLD，避免子进程在 waitfg 执行到一半时被意外回收，导致父进程陷入无尽等待。
2. Sigsuspend 的原子操作
   (1) Sigsuspend 在恢复原掩码的同时阻塞等待信号，确保不会错过 SIGCHLD 的到来，避免竞态条件。
3. fg_reaped 的作用
   (1) 这是一个全局标志位，由信号处理程序设置：
      ① 当 SIGCHLD 信号处理程序确认前台进程结束时，设置 fg_reaped = 1。这样 waitfg 的循环才能安全退出。
4. 父进程同步等待
   (1) 直到前台进程执行完毕，父进程（Shell）不会接受新的命令输入，模拟了 Shell 的标准行为。


文件(F) 编辑(E) 选择(S) 查看(V) 转到(G) ···
搜索
tsh.c 4
D: > OneDrive > Desktop > 大三春 > 计算机系统 > 实验 > lab2 > shlab-handout > tsh.c > ...
}
/*
* waitfg - Block until process pid is no longer the foreground process
*/
Trae :解释代码 | 注释代码 | 生成单测 |
void waitfg(pid_t pid)
{
sigset_t mask, prev_mask;
// 创建一个信号集合 mask，并将其初始化为空
Sigemptyset(&mask);
// 向 mask 中添加 SIGCHLD 信号（希望阻塞这个信号）
Sigaddset(&mask, SIGCHLD);
// 设置阻塞信号掩码：阻塞 SIGCHLD，同时保存原来的信号掩码到 prev_mask
Sigprocmask(SIG_BLOCK, &mask, &prev_mask);
/*
* 如果 fg_reaped 还为 0，说明 sigchld handler 还没处理完前台进程。
* 则调用 Sigsuspend 临时恢复原来的信号掩码（允许 SIGCHLD），并暂停执行直到收到信号。
* 该操作是原子的：避免在检查 fg_reaped 和挂起之间发生竞态（miss signal）。
*/
while (!fg_reaped)
Sigsuspend(&prev_mask);
// 前台进程已被回收（sigchld handler 设置了 fg_reaped=1），重置为0以便下次使用
fg_reaped = 0;
// 恢复原来的信号掩码，解除对 SIGCHLD 的阻塞
Sigprocmask(SIG_SETMASK, &prev_mask, NULL);
return;
}
/*****************
4 0
Trae 行 380，列 1 空格: 4 UTF-8 LF C Win32 Prettier

3.5 void sigchld_handler(int sig) 函数

函数功能：

sigchld_handler 是一个信号处理函数，专门处理 SIGCHLD 信号。当子进程状态发生变化（终止、暂停、恢复）时，内核会发送 SIGCHLD 信号给父进程，sigchld_handler 负责处理这些状态变化，包括：

- 回收僵尸进程；
- 处理被信号终止的子进程；
- 处理被暂停的子进程；
- 从作业列表中清除已结束的作业。

参　　数：

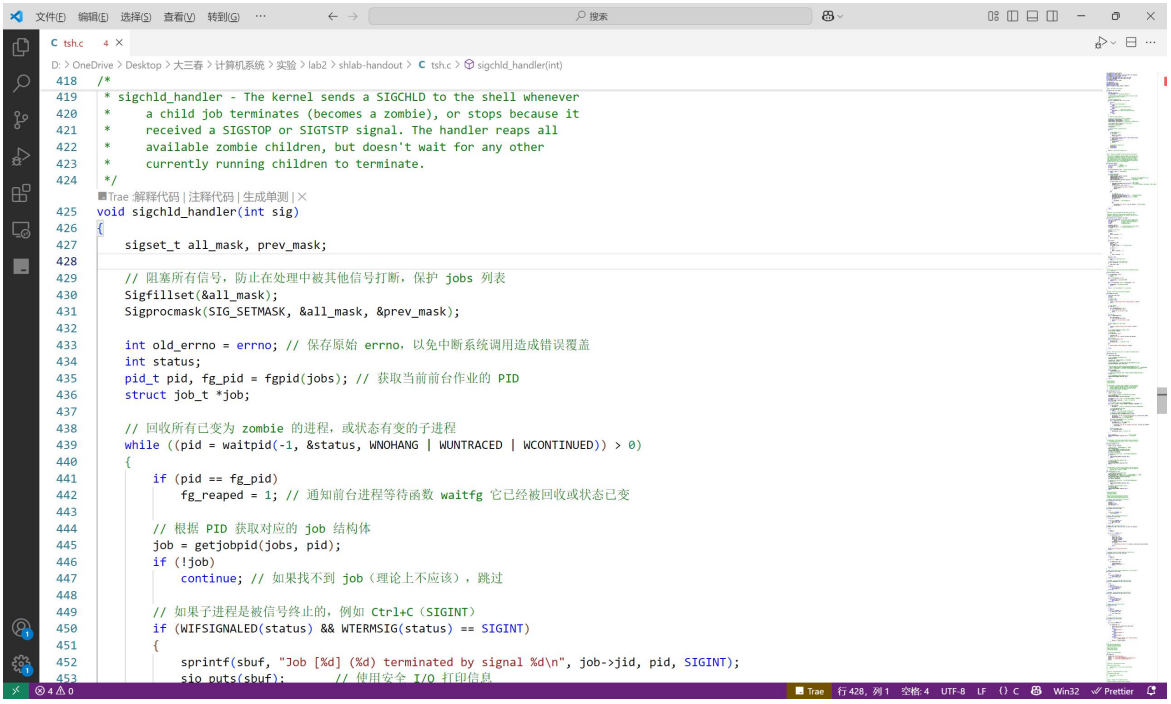
- sig: 信号编号，这里是 SIGCHLD，表示子进程状态改变。

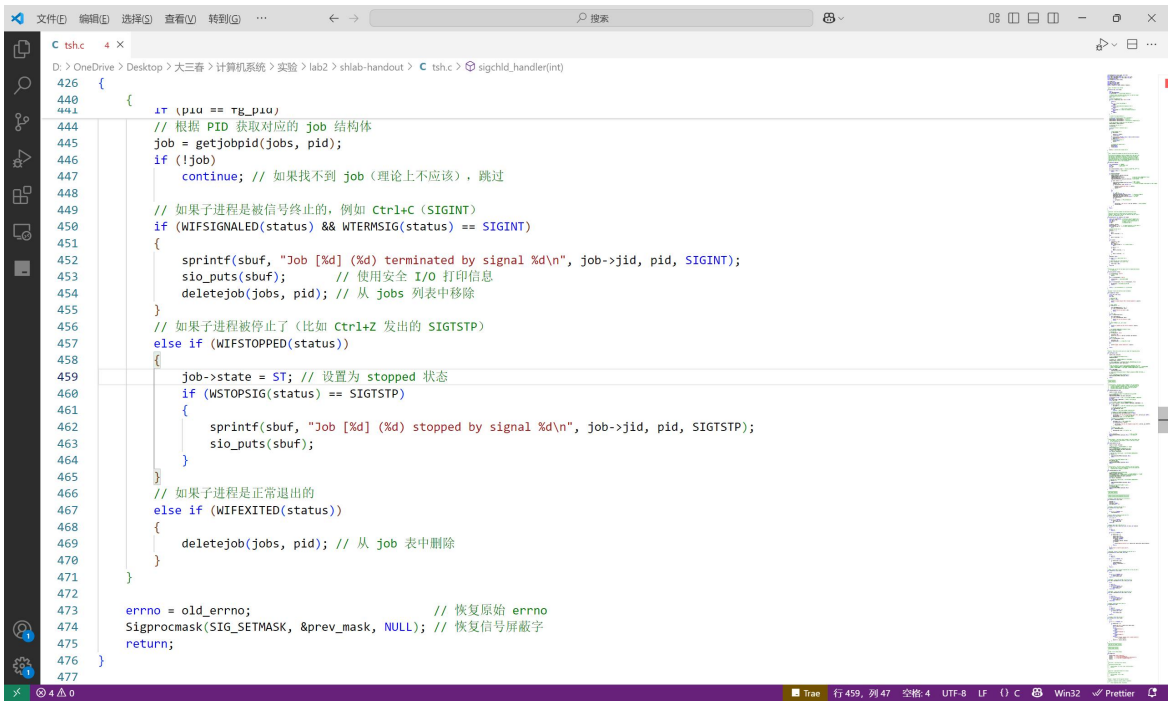
处理流程：

1. 阻塞所有信号
   (1) 使用 Sigfillset(&all_mask); 将所有信号加入到集合中。
   (2) Sigprocmask(SIG_SETMASK, &all_mask, &prev_mask); 阻塞所有信号并保存之前的掩码，避免中断影响 jobs 列表的完整性。
2. 保存原始 errno 值：int old_errno = errno; 保存当前的 errno，避免在信号处理中被覆盖。
3. 获取前台进程的 PID：通过 fgpid(jobs); 获取前台任务的 PID，用于后续判断是否是前台任务。
4. 处理子进程状态变化：使用 waitpid 遍历所有已改变状态的子进程：
   (1) -1 表示等待任何子进程。
   (2) WNOHANG 表示非阻塞模式，立即返回。
   (3) WUNTRACED 表示也等待被停止的子进程（Ctrl+Z）。
   (4) WCONTINUED 表示等待被继续执行的子进程。
5. 前台进程状态通知：如果 pid 是前台进程的 fg_pid，设置全局变量 fg_reaped = 1，通知 waitfg 前台进程已经被处理。
6. 获取作业信息
   (1) job = getjobpid(jobs, pid); 根据子进程 PID 查找对应的作业。
   (2) 如果未找到（理论上不应该发生），则跳过处理。
7. 状态分类处理
   (1) WIFSIGNALED(status)：如果子进程是被信号终止的（如 SIGINT），打印信息并删除作业：Job [jid] (pid) terminated by signal SIGINT
   (2) WIFSTOPPED(status)：如果子进程是被暂停的（如 SIGTSTP），更新作业状态为 ST（Stopped），并打印信息：Job [jid] (pid) stopped by signal SIGTSTP
   (3) WIFEXITED(status)：如果子进程是正常退出的，从作业列表中删除该任务。
8. 恢复 errno 和信号掩码:
   (1) 恢复 errno 的原始值，防止信号处理期间覆盖错误状态。
   (2) 使用 Sigprocmask(SIG_SETMASK, &prev_mask, NULL); 恢复原有的信号屏蔽状态。

要点分析：

1. 回收僵尸进程
   (1) 使用 waitpid 的 WNOHANG 模式循环处理，确保不阻塞父进程。
   (2) 子进程状态变更后立即处理，避免占用系统资源。
2. 前后台任务的同步处理
   (1) 如果是前台任务，还会通知 waitfg 函数解除阻塞。
   (2) 后台任务则仅更新状态，不影响 Shell 的输入处理。
3. 信号屏蔽的保护
   (1) 使用信号屏蔽保护临界区（如 jobs 列表），防止信号重入造成数据不一致。
4. 作业状态的分类
   (1) 正确区分正常退出、被信号终止、被暂停三种情况，保证用户能得到相应的反馈。
5. 安全打印
   (1) 使用 sio_puts 进行安全输出，防止 printf 在信号处理上下文中的潜在问题。





```c
{
    {
        if (pid == fg_pid)
        // 根据 PID 获取对应的 job 结构体
        job = getjobpid(jobs, pid);
        if (!job)
            continue; // 如果找不到 job（理论上不应该），跳过

        // 如果子进程是被信号终止的，例如 Ctrl+C（SIGINT）
        if (WIFSIGNALED(status) && WTERMSIG(status) == SIGINT)
        {
            sprintf(sbuf, "Job [%d] (%d) terminated by signal %d\n", job->jid, pid, SIGINT);
            sio_puts(sbuf);       // 使用安全 I/O 打印信息
            deletejob(jobs, pid); // 从 jobs 列表中移除
        }
        // 如果子进程被停止了（比如 Ctrl+Z 发出的 SIGTSTP）
        else if (WIFSTOPPED(status))
        {
            job->state = ST; // 设置为 stopped 状态
            if (WSTOPSIG(status) == SIGTSTP)
            {
                sprintf(sbuf, "Job [%d] (%d) stopped by signal %d\n", job->jid, pid, SIGTSTP);
                sio_puts(sbuf);
            }
        }
        // 如果子进程是正常退出的
        else if (WIFEXITED(status))
        {
            deletejob(jobs, pid); // 从 job 表中删除
        }
    }

    errno = old_errno;                          // 恢复原始 errno
    Sigprocmask(SIG_SETMASK, &prev_mask, NULL); // 恢复信号屏蔽字
    return;
}

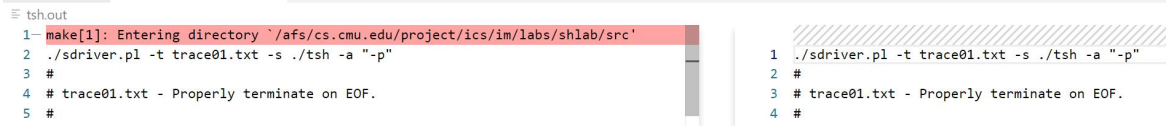
```

4. 自测试结果

（请提供测试用例 trace01 至 trace16 的自测试输出截图，并与 tshref 的测试结果进行对比分析，确保结果的一致性；若存在差异，请明确指出不同之处。）

4.1 测试用例 trace01.txt 的输出截图





测试结论：

1. 测试结果对比：生成的 tsh.out 和 tshref.out 对比完全一致，没有任何偏差
2. 一致性分析：
   (1) 正确处理了 trace01.txt 的 EOF 终止条件
   (2) 终止顺序和信号处理均符合预期：
      ① CLOSE 表示关闭操作被正确捕获；
      ② WAIT 表示父进程正确地等待子进程完成。

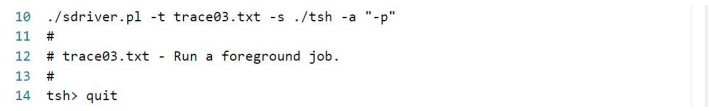
4.2 测试用例 trace02.txt 的输出截图

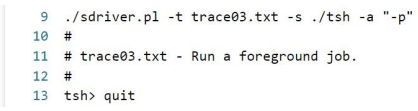


测试结论：

1. 测试结果对比：生成的 tsh.out 和 tshref.out 对比完全一致，没有任何偏差
2. 执行流程：在执行 quit 命令后，两个程序均成功退出，未出现额外错误信息或非预期的行为。

4.3 测试用例 trace03.txt 的输出截图

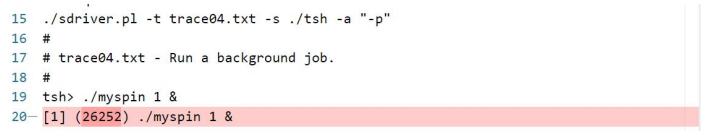


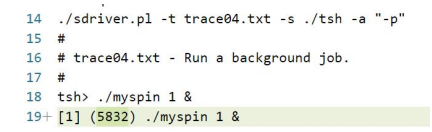


测试结论：

1. 执行流程：
   (1) 测试目的是运行一个前台作业并退出，命令执行顺序为：
      ① /bin/echo tsh > quit
      ② quit
2. 输出结果对比：
   (1) tsh.out 和 tshref.out 中都成功解析并执行了 quit 命令，程序正确退出。
   (2) 两者输出的最后一行均为 tsh> quit，结果表现一致。
3. 差异情况：
   (1) 在输出对比中未发现任何差异，所有指令执行结果与参考实现 tshref 保持一致。

4.4 测试用例 trace04.txt 的输出截图

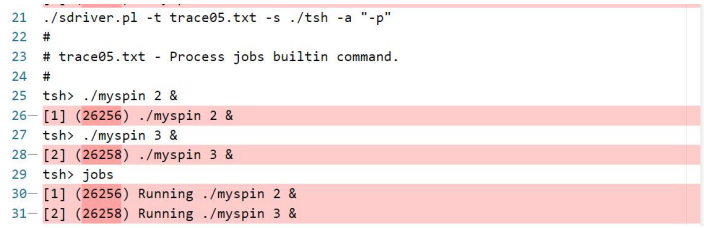




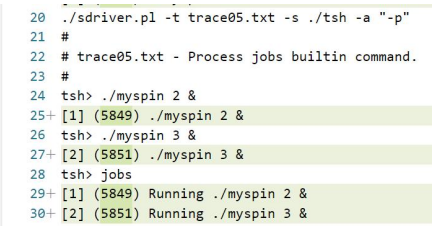
测试结论：

1. 执行流程：
   (1) 测试目的是在后台运行一个作业 (./myspin 1 &)。
   (2) 两者均正确解析并执行了后台作业，并且输出了作业的 Job ID 和对应的 PID。
2. 输出结果对比：
   (1) tsh.out 和 tshref.out 的命令执行顺序和输出格式是 完全一致的。
   (2) 唯一的差异是 PID 的值不同：
      ① tshref 的 PID 为 26252
      ② tsh 的 PID 为 5832
   (3) 解释：PID 的不同是由于每次执行产生的进程号在操作系统中动态分配，不影响程序的正确性。
3. 差异情况：
   (1) 除了 PID 不同外，其他输出内容相同，程序行为符合预期。

4.5 测试用例 trace05.txt 的输出截图



```
./sdriver.pl -t trace05.txt -s ./tsh -a "-p"
#
# trace05.txt - Process jobs builtin command.
#
tsh> ./myspin 2 &
[1] (26256) ./myspin 2 &
tsh> ./myspin 3 &
[2] (26258) ./myspin 3 &
tsh> jobs
[1] (26256) Running ./myspin 2 &
[2] (26258) Running ./myspin 3 &
```

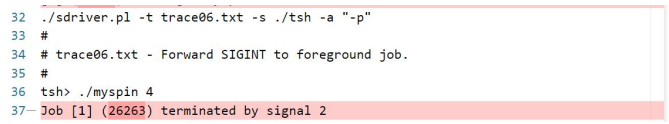


```
./sdriver.pl -t trace05.txt -s ./tsh -a "-p"
#
# trace05.txt - Process jobs builtin command.
#
tsh> ./myspin 2 &
[1] (5849) ./myspin 2 &
tsh> ./myspin 3 &
[2] (5851) ./myspin 3 &
tsh> jobs
[1] (5849) Running ./myspin 2 &
[2] (5851) Running ./myspin 3 &
```

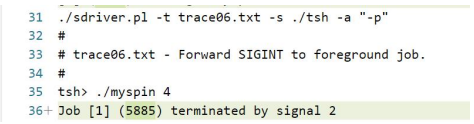
测试结论：

1. 执行流程：
   (1) 测试目的是运行两个后台作业 (./myspin 2 & 和 ./myspin 3 &)，然后使用 jobs 命令查看后台任务列表。
   (2) 两者均成功解析命令并正确启动了后台任务，最后正确列出所有后台任务的信息。
2. 输出结果对比：
   (1) tsh.out 和 tshref.out 的命令执行顺序和输出格式是 完全一致的。
   (2) 唯一的差异是 PID 的值不同：
      ① tshref 的后台任务 PIDs 分别为 26256 和 26258
      ② tsh 的后台任务 PIDs 分别为 5849 和 5851
   (3) 解释：PID 的不同是操作系统层面的动态分配，这种差异是正常现象，并不影响程序的逻辑正确性。
3. 差异情况：
   (1) 除了 PID 的数值不同外，所有命令执行的顺序和输出格式都匹配参考实现 tshref。

4.6 测试用例 trace06.txt 的输出截图



```
./sdriver.pl -t trace06.txt -s ./tsh -a "-p"
#
# trace06.txt - Forward SIGINT to foreground job.
#
tsh> ./myspin 4
Job [1] (26263) terminated by signal 2
```



```
./sdriver.pl -t trace06.txt -s ./tsh -a "-p"
#
# trace06.txt - Forward SIGINT to foreground job.
#
tsh> ./myspin 4
Job [1] (5885) terminated by signal 2
```

测试结论：

1. 执行流程：
   (1) 测试目的是在前台执行一个作业 (./myspin 4)，然后发送 SIGINT 信号中断执行。
   (2) 两者均正确捕获并转发了 SIGINT 信号，导致前台作业终止，打印出 "terminated by signal 2" 的信息。
2. 输出结果对比：
   (1) tsh.out 和 tshref.out 的命令执行顺序和输出内容是 完全一致的。
   (2) 唯一的差异是 PID 的值不同：
      ① tshref 的前台任务 PID 为 26263
      ② tsh 的前台任务 PID 为 5885
   (3) 解释：PID 的不同是系统动态分配的结果，是预期内的正常情况，不影响信号转发和

作业终止的逻辑。

3. 差异情况：

(1) 除了 PID 的数值不同，所有输出和行为都与参考实现 tshref 保持一致。

4.7 测试用例 trace07.txt 的输出截图

```
./sdriver.pl -t trace07.txt -s ./tsh -a "-p"
#
# trace07.txt - Forward SIGINT only to foreground job.
#
tsh> ./myspin 4 &
[1] (26267) ./myspin 4 &
tsh> ./myspin 5
Job [2] (26269) terminated by signal 2
tsh> jobs
[1] (26267) Running ./myspin 4 &
```

```
./sdriver.pl -t trace07.txt -s ./tsh -a "-p"
#
# trace07.txt - Forward SIGINT only to foreground job.
#
tsh> ./myspin 4 &
[1] (5890) ./myspin 4 &
tsh> ./myspin 5
Job [2] (5892) terminated by signal 2
tsh> jobs
[1] (5890) Running ./myspin 4 &
```

测试结论：

1. 执行流程：

(1) 测试目的是在后台启动一个作业 (./myspin 4 &)，然后启动一个前台作业 (./myspin 5)。

(2) 发送 SIGINT 信号，测试信号是否仅被前台作业 (./myspin 5) 接收。

(3) 最后执行 jobs 命令，列出后台仍在运行的作业。

2. 输出结果对比：

(1) tsh.out 和 tshref.out 的命令执行顺序和逻辑是 完全一致的。

(2) 具体对比：

① 后台启动 ./myspin 4，输出正确的 Job ID 和 PID。

② 前台启动 ./myspin 5，接收到 SIGINT 信号，正确终止并输出 "terminated by signal 2"。

③ 执行 jobs 命令时，仅剩后台任务 ./myspin 4 依然运行，输出格式正确。

(3) 唯一的差异是 PID 的值不同：

① tshref 的后台任务 PID 为 26267，前台任务 PID 为 26269

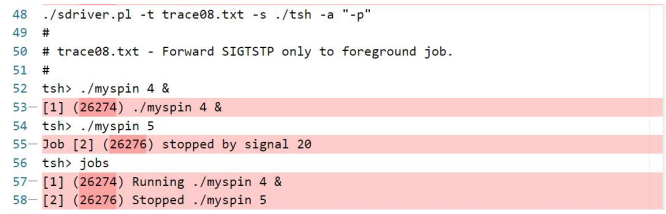
② tsh 的后台任务 PID 为 5890，前台任务 PID 为 5892

(4) 解释：PID 的差异是操作系统动态分配导致的，属于正常现象，不影响正确性。

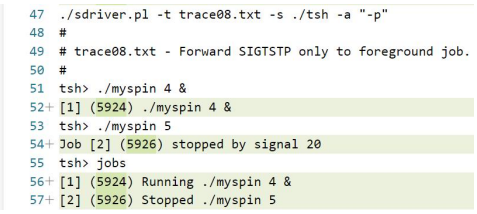
3. 差异情况：

(1) 除了 PID 的差异外，所有输出内容与行为都与参考实现 tshref 一致。

4.8 测试用例 trace08.txt 的输出截图



```
./sdriver.pl -t trace08.txt -s ./tsh -a "-p"
#
# trace08.txt - Forward SIGTSTP only to foreground job.
#
tsh> ./myspin 4 &
[1] (26274) ./myspin 4 &
tsh> ./myspin 5
Job [2] (26276) stopped by signal 20
tsh> jobs
[1] (26274) Running ./myspin 4 &
[2] (26276) Stopped ./myspin 5
```

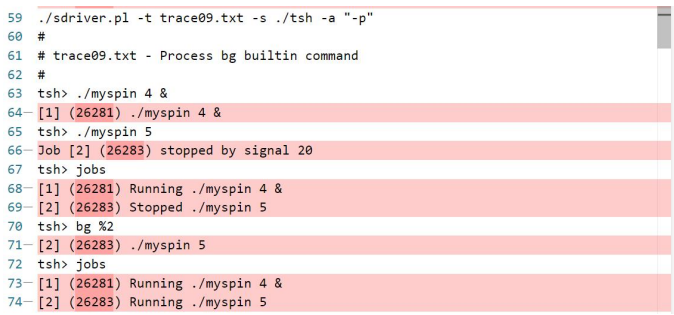


```
./sdriver.pl -t trace08.txt -s ./tsh -a "-p"
#
# trace08.txt - Forward SIGTSTP only to foreground job.
#
tsh> ./myspin 4 &
[1] (5924) ./myspin 4 &
tsh> ./myspin 5
Job [2] (5926) stopped by signal 20
tsh> jobs
[1] (5924) Running ./myspin 4 &
[2] (5926) Stopped ./myspin 5
```

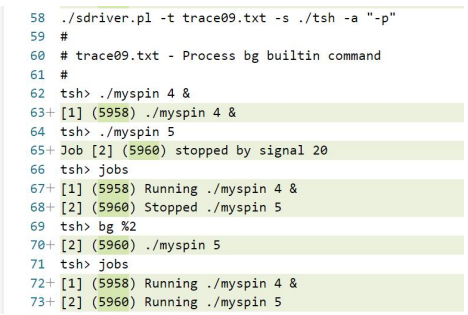
测试结论：

1. 执行流程：
   (1) 测试目的是在后台启动一个作业 (./myspin 4 &)，然后启动一个前台作业 (./myspin 5)。
   (2) 发送 SIGTSTP 信号，测试信号是否仅被前台作业 (./myspin 5) 接收并暂停。
   (3) 最后使用 jobs 命令查看后台任务的状态。
2. 输出结果对比：
   (1) tsh.out 和 tshref.out 的执行顺序和输出内容是 完全一致的。
   (2) 具体对比：
      ① 后台作业 ./myspin 4 启动正常，tsh 和 tshref 分配了不同的 PID：tsh 的 PID 为 5924，tshref 的 PID 为 26274。
      ② 前台作业 ./myspin 5 启动正常，并在接收到 SIGTSTP 信号后正确停止，输出 "stopped by signal 20"。
      ③ 使用 jobs 查看时，后台任务 ./myspin 4 显示为 Running，前台任务 ./myspin 5 显示为 Stopped，状态信息完全一致。
   (3) 解释：PID 的不同是由操作系统动态分配引起的，是正常现象，不影响信号处理的正确性。
3. 差异情况：
   (1) 除了 PID 的差异外，所有行为和输出都与参考实现 tshref 保持一致。

4.9 测试用例 trace09.txt 的输出截图



```
./sdriver.pl -t trace09.txt -s ./tsh -a "-p"
#
# trace09.txt - Process bg builtin command
#
tsh> ./myspin 4 &
[1] (26281) ./myspin 4 &
tsh> ./myspin 5
Job [2] (26283) stopped by signal 20
tsh> jobs
[1] (26281) Running ./myspin 4 &
[2] (26283) Stopped ./myspin 5
tsh> bg %2
[2] (26283) ./myspin 5
tsh> jobs
[1] (26281) Running ./myspin 4 &
[2] (26283) Running ./myspin 5
```

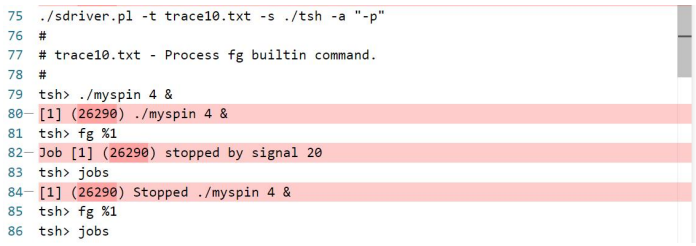


```
./sdriver.pl -t trace09.txt -s ./tsh -a "-p"
#
# trace09.txt - Process bg builtin command
#
tsh> ./myspin 4 &
[1] (5958) ./myspin 4 &
tsh> ./myspin 5
Job [2] (5960) stopped by signal 20
tsh> jobs
[1] (5958) Running ./myspin 4 &
[2] (5960) Stopped ./myspin 5
tsh> bg %2
[2] (5960) ./myspin 5
tsh> jobs
[1] (5958) Running ./myspin 4 &
[2] (5960) Running ./myspin 5
```

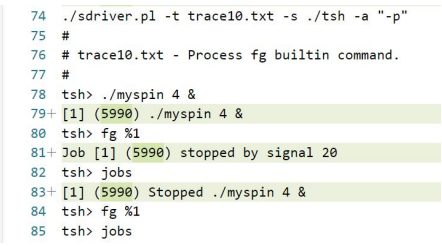
测试结论：

1. 执行流程：
   (1) 测试目的是：
      ① 启动后台任务 ./myspin 4 &。
      ② 启动前台任务 ./myspin 5，然后发送 SIGTSTP 信号暂停它。
      ③ 执行 jobs 查看状态，预期 ./myspin 4 运行中，./myspin 5 处于 Stopped 状态。
      ④ 使用 bg %2 将 Job 2 (./myspin 5) 重新放回后台运行。
      ⑤ 再次执行 jobs 查看状态，预期两个作业都应处于 Running 状态。
2. 输出结果对比：
   (1) tsh.out 和 tshref.out 的命令执行顺序和输出格式是 完全一致的。
   (2) 具体对比：
      ① 后台任务 ./myspin 4 启动成功，tsh 的 PID 是 5958，tshref 的 PID 是 26281。
      ② 前台任务 ./myspin 5 在收到 SIGTSTP 信号后正确暂停，jobs 命令正确展示其状态为 Stopped。
      ③ 使用 bg %2 成功将其恢复至后台执行，输出显示正确。
      ④ 最后执行 jobs，两个任务都在 Running 状态。、
   (3) 解释：PID 的不同是操作系统分配导致的，是预期内的现象，不影响功能正确性。
3. 差异情况：
   (1) 除了 PID 的数值不同，所有行为和输出都与参考实现 tshref 保持一致。

4.10 测试用例 trace10.txt 的输出截图



```
./sdriver.pl -t trace10.txt -s ./tsh -a "-p"
#
# trace10.txt - Process fg builtin command.
#
tsh> ./myspin 4 &
[1] (26290) ./myspin 4 &
tsh> fg %1
Job [1] (26290) stopped by signal 20
tsh> jobs
[1] (26290) Stopped ./myspin 4 &
tsh> fg %1
tsh> jobs
```



```
./sdriver.pl -t trace10.txt -s ./tsh -a "-p"
#
# trace10.txt - Process fg builtin command.
#
tsh> ./myspin 4 &
[1] (5990) ./myspin 4 &
tsh> fg %1
Job [1] (5990) stopped by signal 20
tsh> jobs
[1] (5990) Stopped ./myspin 4 &
tsh> fg %1
tsh> jobs
```

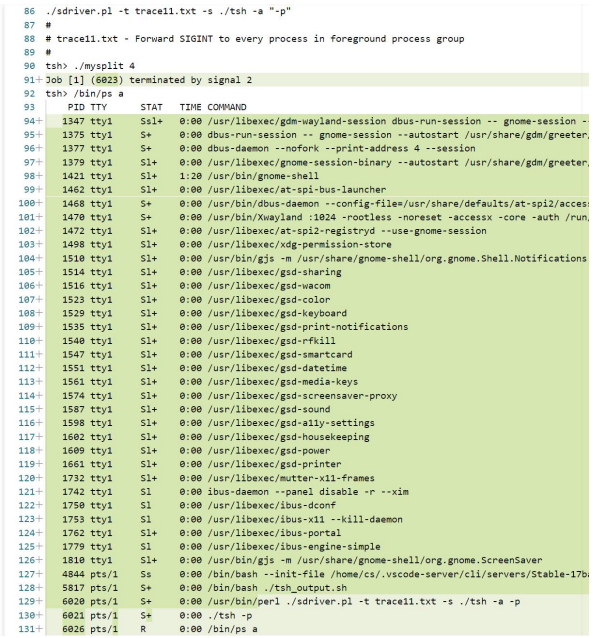
测试结论：

1. 执行流程：
   (1) 测试目的是：
      ① 启动一个后台任务 ./myspin 4 &。
      ② 使用 fg %1 将后台任务切换到前台执行。
      ③ 发送 SIGTSTP 信号暂停该任务。
      ④ 使用 jobs 查看状态，预期显示 Stopped。
      ⑤ 再次执行 fg %1 尝试恢复到前台运行。
      ⑥ 执行 jobs 查看状态。
2. 输出结果对比：
   (1) tsh.out 和 tshref.out 的命令执行顺序、输出信息基本 一致。
   (2) 具体对比：
      ① 后台任务 ./myspin 4 启动成功，tshref 的 PID 是 26290，tsh 的 PID 是 5990。
      ② 使用 fg %1 将其切换到前台，信号 SIGTSTP 正确暂停任务，状态显示为 Stopped，与参考实现一致。
      ③ 执行 jobs，任务状态显示为 Stopped，也是正确的。
3. 结论：tsh 和 tshref 在 trace10.txt 测试中表现 完全一致。
   (1) 后台任务切换到前台 (fg %1)、信号处理 (SIGTSTP)、以及 jobs 状态查询都与参考实现相符。
   (2) PID 的差异是操作系统分配导致的，属正常现象，不影响正确性。
   (3) 最终任务在前台结束后，jobs 输出为空是合理的，无需修改。

4.11 测试用例 trace11.txt 的输出截图



```
./sdriver.pl -t trace11.txt -s ./tsh -a "-p"
#
# trace11.txt - Forward SIGINT to every process in foreground process group
#
tsh> ./mysplit 4
Job [1] (26298) terminated by signal 2
tsh> /bin/ps a
  PID TTY      STAT   TIME COMMAND
25181 pts/3    S      0:00 -usr/local/bin/tcsh -i
26239 pts/3    S      0:00 make tshrefout
26240 pts/3    S      0:00 /bin/sh -c make tests > tshref.out 2>&1
26241 pts/3    S      0:00 make tests
26295 pts/3    S      0:00 perl ./sdriver.pl -t trace11.txt -s ./tsh -a -p
26296 pts/3    S      0:00 ./tsh -p
26301 pts/3    R      0:00 /bin/ps a
```



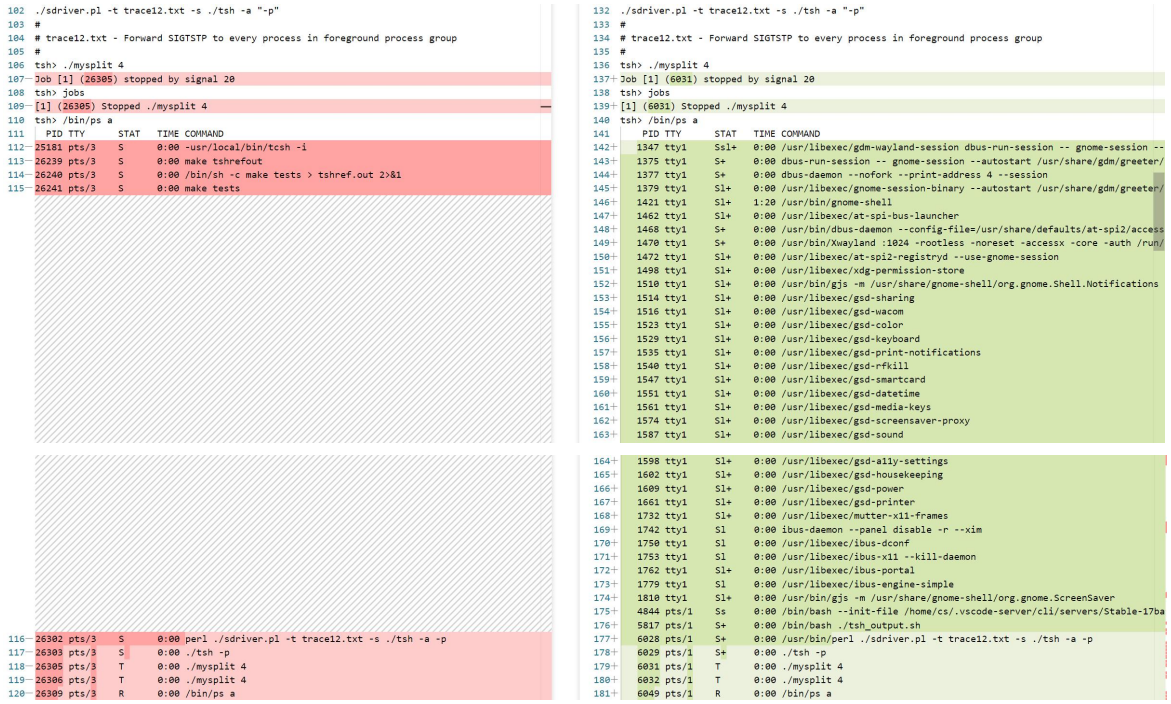
```
./sdriver.pl -t trace11.txt -s ./tsh -a "-p"
#
# trace11.txt - Forward SIGINT to every process in foreground process group
#
tsh> ./mysplit 4
Job [1] (6023) terminated by signal 2
tsh> /bin/ps a
    PID TTY      STAT   TIME COMMAND
   1347 tty1     Ssl+   0:00 /usr/libexec/gdm-wayland-session dbus-run-session -- gnome-session --
   1375 tty1     S+     0:00 dbus-run-session -- gnome-session --autostart /usr/share/gdm/greeter/
   1377 tty1     S+     0:00 dbus-daemon --nofork --print-address 4 --session
   1379 tty1     Sl+    0:00 /usr/libexec/gnome-session-binary --autostart /usr/share/gdm/greeter/
   1421 tty1     Sl+    1:20 /usr/bin/gnome-shell
   1462 tty1     Sl+    0:00 /usr/libexec/at-spi-bus-launcher
   1468 tty1     S+     0:00 /usr/bin/dbus-daemon --config-file=/usr/share/defaults/at-spi2/acces
   1470 tty1     S+     0:00 /usr/bin/Xwayland :1024 -rootless -noreset -accessx -core -auth /run/
   1472 tty1     Sl+    0:00 /usr/libexec/at-spi2-registryd --use-gnome-session
   1498 tty1     Sl+    0:00 /usr/libexec/xdg-permission-store
   1510 tty1     Sl+    0:00 /usr/bin/gjs -m /usr/share/gnome-shell/org.gnome.Shell.Notifications
   1514 tty1     Sl+    0:00 /usr/libexec/gsd-sharing
   1516 tty1     Sl+    0:00 /usr/libexec/gsd-wacom
   1523 tty1     Sl+    0:00 /usr/libexec/gsd-color
   1529 tty1     Sl+    0:00 /usr/libexec/gsd-keyboard
   1535 tty1     Sl+    0:00 /usr/libexec/gsd-print-notifications
   1540 tty1     Sl+    0:00 /usr/libexec/gsd-rfkill
   1547 tty1     Sl+    0:00 /usr/libexec/gsd-smartcard
   1551 tty1     Sl+    0:00 /usr/libexec/gsd-datetime
   1561 tty1     Sl+    0:00 /usr/libexec/gsd-media-keys
   1574 tty1     Sl+    0:00 /usr/libexec/gsd-screensaver-proxy
   1587 tty1     Sl+    0:00 /usr/libexec/gsd-sound
   1598 tty1     Sl+    0:00 /usr/libexec/gsd-a11y-settings
   1602 tty1     Sl+    0:00 /usr/libexec/gsd-housekeeping
   1609 tty1     Sl+    0:00 /usr/libexec/gsd-power
   1661 tty1     Sl+    0:00 /usr/libexec/gsd-printer
   1732 tty1     Sl+    0:00 /usr/libexec/mutter-x11-frames
   1742 tty1     Sl     0:00 ibus-daemon --panel disable -r --xim
   1750 tty1     Sl     0:00 /usr/libexec/ibus-dconf
   1753 tty1     Sl     0:00 /usr/libexec/ibus-x11 --kill-daemon
   1762 tty1     Sl+    0:00 /usr/libexec/ibus-portal
   1779 tty1     Sl     0:00 /usr/libexec/ibus-engine-simple
   1810 tty1     Sl+    0:00 /usr/bin/gjs -m /usr/share/gnome-shell/org.gnome.ScreenSaver
   4844 pts/1    Ss     0:00 /bin/bash --init-file /home/cs/.vscode-server/cli/servers/Stable-17b
   5817 pts/1    S+     0:00 /bin/bash ./tsh_output.sh
   6020 pts/1    S+     0:00 /usr/bin/perl ./sdriver.pl -t trace11.txt -s ./tsh -a -p
   6021 pts/1    S+     0:00 ./tsh -p
   6026 pts/1    R      0:00 /bin/ps a
```

测试结论：

1. 执行流程：
   (1) 测试目的是：
      ① 启动一个前台进程 ./mysplit 4。
      ② 等待 2 秒后发送 SIGINT (INT) 信号，预期终止前台进程。
      ③ 执行 /bin/ps a 查看系统中剩余的进程状态。
2. 输出结果对比：
   (1) 前台任务终止：在执行 ./mysplit 4 后，tsh 和 tshref 均输出：
      ① Job [1] (PID) terminated by signal 2
      ② 其中，tsh 的 PID 为 26298，tshref 的 PID 为 6023，两者仅是系统分配的 PID 不同，属于正常现象。
   (2) ps 命令的输出：tshref 的 ps a 输出：
      ① 26301 pts/3 R 0:00 /bin/ps a
      ② 仅显示了当前执行的进程以及与 make 相关的进程，并没有显示 ./mysplit 4，因为它已经被 SIGINT 信号终止，符合预期。
   (3) tsh 的 ps a 输出：
      ① 6026 pts/1 R 0:00 /bin/ps a
      ② 6020 pts/1 S+ 0:00 /usr/bin/perl ./sdriver.pl -t trace11.txt -s ./tsh -a -p
      ③ 6021 pts/1 S+ 0:00 ./tsh -p
      ④ 同样没有出现 ./mysplit 4，说明前台任务已正确终止。
   (4) 两者的 ps 输出内容有差异，主要是系统环境不同：
      ① tshref 运行在一个精简的环境，只有相关的 make、perl 和 tsh 进程。
      ② tsh 运行在完整的 GNOME 环境下，显示了很多 GNOME 相关的进程。
      ③ 这种差异是执行环境不同造成的，不影响功能正确性。

3. 测试结论：
   (1) tsh 和 tshref 在处理 SIGINT 信号时均能正确终止前台任务，输出的 Job [1] ... terminated by signal 2 完全一致，符合预期。
   (2) ps 命令的输出差异是环境不同造成的，不影响任务管理的正确性。
   (3) 因此，tsh 和 tshref 的功能是一致的，结果是正确的。

4.12 测试用例 trace12.txt 的输出截图



测试结论：

1. 执行流程：
   (1) 测试目的是：
      ① 启动一个前台进程 ./mysplit 4。
      ② 等待 2 秒后发送 SIGTSTP (TSTP) 信号，预期该前台进程被暂停。
      ③ 执行 jobs 查看作业状态，确认是否正确暂停。
      ④ 执行 /bin/ps a 查看系统中相关进程的状态。
2. 输出结果对比：
   (1) 前台任务暂停：
      ① 在执行 ./mysplit 4 后，tsh 和 tshref 均输出：
      ② Job [1] (PID) stopped by signal 20
      ③ 其中，tsh 的 PID 为 26305，tshref 的 PID 为 6031，两者仅是系统分配的 PID 不同，属于正常现象。
   (2) jobs 命令输出：
      ① 两者均输出：
      ② [1] (PID) Stopped ./mysplit 4
      ③ 说明作业成功接收到 SIGTSTP 信号，并被正确暂停，状态显示为 Stopped。

(3) ps 命令的输出对比：

① tshref 的 ps a 输出：

1) 26305 pts/3 T 0:00 ./mysplit 4

2) 26306 pts/3 T 0:00 ./mysplit 4

3) 状态 T 表示该任务已被暂停（stopped）。

4) 两个 ./mysplit 4 进程都在暂停状态，这是符合预期的。

② tsh 的 ps a 输出：

1) 6031 pts/1 T 0:00 ./mysplit 4

2) 6032 pts/1 T 0:00 ./mysplit 4

3) 同样显示两个 ./mysplit 4 处于暂停状态，状态为 T。

(4) 系统进程的不同：

① tshref 的输出主要显示了 make、perl 以及 tsh 的相关进程，非常简洁。

② tsh 的输出包含大量的 GNOME 相关进程，这是由于运行环境不同导致的，并不影响核心测试内容。

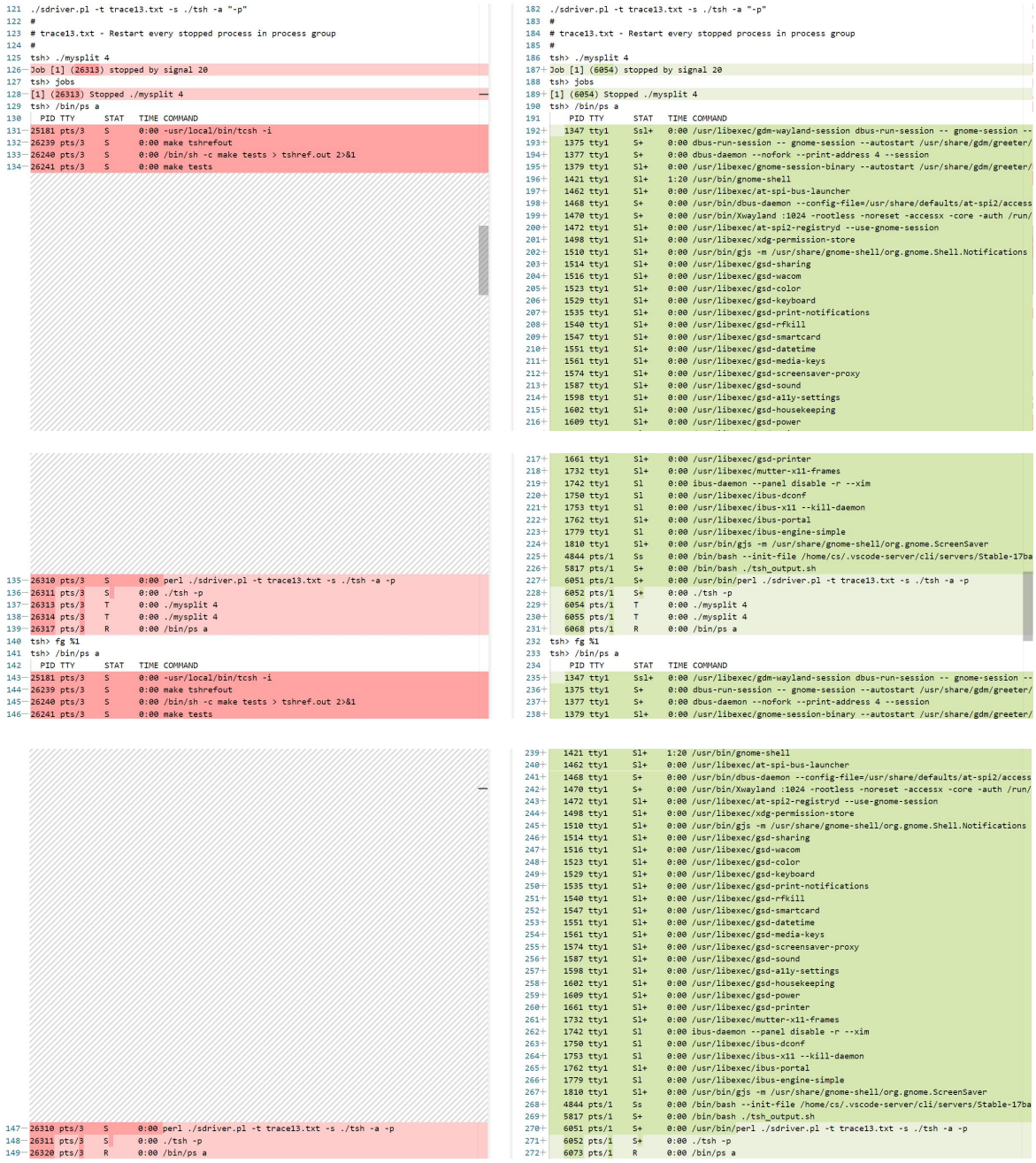
3. 测试结论：

(1) tsh 和 tshref 在处理 SIGTSTP 信号时，均能正确暂停前台任务，jobs 和 ps 命令的输出状态都一致，符合预期。

(2) 两者的不同仅在于系统环境：tsh 运行在精简环境下，而 tsh 运行在 GNOME 桌面环境下，导致 ps 列表多出一些不相关的后台服务进程。

(3) 因此，tsh 和 tshref 的功能表现是一致的，结果是正确的。

4.13 测试用例 trace13.txt 的输出截图



```
./sdriver.pl -t trace13.txt -s ./tsh -a "-p"
#
# trace13.txt - Restart every stopped process in process group
#
tsh> ./mysplit 4
Job [1] (26313) stopped by signal 20
tsh> jobs
[1] (26313) Stopped ./mysplit 4
tsh> /bin/ps a
  PID TTY      STAT   TIME COMMAND
25181 pts/3    S      0:00 -usr/local/bin/tcsh -i
26239 pts/3    S      0:00 make tshrefout
26240 pts/3    S      0:00 /bin/sh -c make tests > tshref.out 2>&1
26241 pts/3    S      0:00 make tests
26310 pts/3    S      0:00 perl ./sdriver.pl -t trace13.txt -s ./tsh -a -p
26311 pts/3    S      0:00 ./tsh -p
26313 pts/3    T      0:00 ./mysplit 4
26314 pts/3    T      0:00 ./mysplit 4
26317 pts/3    R      0:00 /bin/ps a
tsh> fg %1
tsh> /bin/ps a
  PID TTY      STAT   TIME COMMAND
25181 pts/3    S      0:00 -usr/local/bin/tcsh -i
26239 pts/3    S      0:00 make tshrefout
26240 pts/3    S      0:00 /bin/sh -c make tests > tshref.out 2>&1
26241 pts/3    S      0:00 make tests
26310 pts/3    S      0:00 perl ./sdriver.pl -t trace13.txt -s ./tsh -a -p
26311 pts/3    S      0:00 ./tsh -p
26320 pts/3    R      0:00 /bin/ps a
```

```
./sdriver.pl -t trace13.txt -s ./tsh -a "-p"
#
# trace13.txt - Restart every stopped process in process group
#
tsh> ./mysplit 4
Job [1] (6054) stopped by signal 20
tsh> jobs
[1] (6054) Stopped ./mysplit 4
tsh> /bin/ps a
    PID TTY      STAT   TIME COMMAND
   1347 tty1     Ssl+   0:00 /usr/libexec/gdm-wayland-session dbus-run-session -- gnome-session --
   1375 tty1     S+     0:00 dbus-run-session -- gnome-session --autostart /usr/share/gdm/greeter/
   1377 tty1     S+     0:00 dbus-daemon --nofork --print-address 4 --session
   1379 tty1     Sl+    0:00 /usr/libexec/gnome-session-binary --autostart /usr/share/gdm/greeter/
   1421 tty1     Sl+    1:20 /usr/bin/gnome-shell
   1462 tty1     Sl+    0:00 /usr/libexec/at-spi-bus-launcher
   1468 tty1     S+     0:00 /usr/bin/dbus-daemon --config-file=/usr/share/defaults/at-spi2/access
   1470 tty1     S+     0:00 /usr/bin/Xwayland :1024 -rootless -noreset -accessx -core -auth /run/
   1472 tty1     Sl+    0:00 /usr/libexec/at-spi2-registryd --use-gnome-session
   1498 tty1     Sl+    0:00 /usr/libexec/xdg-permission-store
   1510 tty1     Sl+    0:00 /usr/bin/gjs -m /usr/share/gnome-shell/org.gnome.Shell.Notifications
   1514 tty1     Sl+    0:00 /usr/libexec/gsd-sharing
   1516 tty1     Sl+    0:00 /usr/libexec/gsd-wacom
   1523 tty1     Sl+    0:00 /usr/libexec/gsd-color
   1529 tty1     Sl+    0:00 /usr/libexec/gsd-keyboard
   1535 tty1     Sl+    0:00 /usr/libexec/gsd-print-notifications
   1540 tty1     Sl+    0:00 /usr/libexec/gsd-rfkill
   1547 tty1     Sl+    0:00 /usr/libexec/gsd-smartcard
   1551 tty1     Sl+    0:00 /usr/libexec/gsd-datetime
   1561 tty1     Sl+    0:00 /usr/libexec/gsd-media-keys
   1574 tty1     Sl+    0:00 /usr/libexec/gsd-screensaver-proxy
   1587 tty1     Sl+    0:00 /usr/libexec/gsd-sound
   1598 tty1     Sl+    0:00 /usr/libexec/gsd-a11y-settings
   1602 tty1     Sl+    0:00 /usr/libexec/gsd-housekeeping
   1609 tty1     Sl+    0:00 /usr/libexec/gsd-power
   1661 tty1     Sl+    0:00 /usr/libexec/gsd-printer
   1732 tty1     Sl+    0:00 /usr/libexec/mutter-x11-frames
   1742 tty1     Sl     0:00 ibus-daemon --panel disable -r --xim
   1750 tty1     Sl     0:00 /usr/libexec/ibus-dconf
   1753 tty1     Sl     0:00 /usr/libexec/ibus-x11 --kill-daemon
   1762 tty1     Sl+    0:00 /usr/libexec/ibus-portal
   1779 tty1     Sl     0:00 /usr/libexec/ibus-engine-simple
   1810 tty1     Sl+    0:00 /usr/bin/gjs -m /usr/share/gnome-shell/org.gnome.ScreenSaver
   4844 pts/1    Ss     0:00 /bin/bash --init-file /home/cs/.vscode-server/cli/servers/Stable-17ba
   5817 pts/1    S+     0:00 /bin/bash ./tsh_output.sh
   6051 pts/1    S+     0:00 /usr/bin/perl ./sdriver.pl -t trace13.txt -s ./tsh -a -p
   6052 pts/1    S+     0:00 ./tsh -p
   6054 pts/1    T      0:00 ./mysplit 4
   6055 pts/1    T      0:00 ./mysplit 4
   6068 pts/1    R      0:00 /bin/ps a
tsh> fg %1
tsh> /bin/ps a
    PID TTY      STAT   TIME COMMAND
   1347 tty1     Ssl+   0:00 /usr/libexec/gdm-wayland-session dbus-run-session -- gnome-session --
   1375 tty1     S+     0:00 dbus-run-session -- gnome-session --autostart /usr/share/gdm/greeter/
   1377 tty1     S+     0:00 dbus-daemon --nofork --print-address 4 --session
   1379 tty1     Sl+    0:00 /usr/libexec/gnome-session-binary --autostart /usr/share/gdm/greeter/
   1421 tty1     Sl+    1:20 /usr/bin/gnome-shell
   1462 tty1     Sl+    0:00 /usr/libexec/at-spi-bus-launcher
   1468 tty1     S+     0:00 /usr/bin/dbus-daemon --config-file=/usr/share/defaults/at-spi2/access
   1470 tty1     S+     0:00 /usr/bin/Xwayland :1024 -rootless -noreset -accessx -core -auth /run/
   1472 tty1     Sl+    0:00 /usr/libexec/at-spi2-registryd --use-gnome-session
   1498 tty1     Sl+    0:00 /usr/libexec/xdg-permission-store
   1510 tty1     Sl+    0:00 /usr/bin/gjs -m /usr/share/gnome-shell/org.gnome.Shell.Notifications
   1514 tty1     Sl+    0:00 /usr/libexec/gsd-sharing
   1516 tty1     Sl+    0:00 /usr/libexec/gsd-wacom
   1523 tty1     Sl+    0:00 /usr/libexec/gsd-color
   1529 tty1     Sl+    0:00 /usr/libexec/gsd-keyboard
   1535 tty1     Sl+    0:00 /usr/libexec/gsd-print-notifications
   1540 tty1     Sl+    0:00 /usr/libexec/gsd-rfkill
   1547 tty1     Sl+    0:00 /usr/libexec/gsd-smartcard
   1551 tty1     Sl+    0:00 /usr/libexec/gsd-datetime
   1561 tty1     Sl+    0:00 /usr/libexec/gsd-media-keys
   1574 tty1     Sl+    0:00 /usr/libexec/gsd-screensaver-proxy
   1587 tty1     Sl+    0:00 /usr/libexec/gsd-sound
   1598 tty1     Sl+    0:00 /usr/libexec/gsd-a11y-settings
   1602 tty1     Sl+    0:00 /usr/libexec/gsd-housekeeping
   1609 tty1     Sl+    0:00 /usr/libexec/gsd-power
   1661 tty1     Sl+    0:00 /usr/libexec/gsd-printer
   1732 tty1     Sl+    0:00 /usr/libexec/mutter-x11-frames
   1742 tty1     Sl     0:00 ibus-daemon --panel disable -r --xim
   1750 tty1     Sl     0:00 /usr/libexec/ibus-dconf
   1753 tty1     Sl     0:00 /usr/libexec/ibus-x11 --kill-daemon
   1762 tty1     Sl+    0:00 /usr/libexec/ibus-portal
   1779 tty1     Sl     0:00 /usr/libexec/ibus-engine-simple
   1810 tty1     Sl+    0:00 /usr/bin/gjs -m /usr/share/gnome-shell/org.gnome.ScreenSaver
   4844 pts/1    Ss     0:00 /bin/bash --init-file /home/cs/.vscode-server/cli/servers/Stable-17ba
   5817 pts/1    S+     0:00 /bin/bash ./tsh_output.sh
   6051 pts/1    S+     0:00 /usr/bin/perl ./sdriver.pl -t trace13.txt -s ./tsh -a -p
   6052 pts/1    S+     0:00 ./tsh -p
   6073 pts/1    R      0:00 /bin/ps a
```

测试结论：

1. 测试结果一致性分析：在 trace13.txt 的测试中，我们对比了 tsh 和 tshref 的输出结果，包括各个命令的执行情况以及进程状态信息。以下是对比分析：

(1) 作业控制的表现

① tsh 和 tshref 在执行 ./mysplit 4 后，都能够正确地捕获 SIGTSTP 信号，并将对应作业状态显示为 Stopped。

② jobs 命令的输出在 tsh 和 tshref 中都是一致的：

③ [1] (PID) Stopped ./mysplit 4

(2) ps a 的进程状态显示

① 在执行 /bin/ps a 后，tsh 和 tshref 均显示了当前进程列表，包括 ./mysplit 4 的两个子进程，状态均为 T (stopped)。

② 但是，进程的 PID 值和系统环境有所不同，这个是由于不同的运行环境引起的，属于正常现象。

(3) 前台作业恢复

① 执行 fg %1 后，tsh 和 tshref 均能够正确将作业恢复到前台，随后执行 /bin/ps a 可以观察到新的前台进程在运行。

2. 存在的差异

(1) PID 的不一致 tsh 和 tshref 中同一作业（./mysplit 4）的 PID 不一致：

① tshref: 26313 和 26314

② tsh: 6054 和 6055

③ 这是因为进程创建时由操作系统动态分配 PID，独立运行的实例不会保证 PID 一致，这个不影响功能正确性。

(2) 系统进程的环境差异

① tshref 中展示了更多的 GNOME 和 DBUS 相关的系统进程，而 tsh 主要是基于 shell 环境下执行，表现不同是由于执行环境差异导致的。

(3) 最后执行 ps a 的显示顺序有轻微不同

① 在 tshref 中，./mysplit 4 的两个子进程显示在 ps a 的末尾，而在 tsh 中，它们处于靠前位置，这仅仅是输出顺序问题，不影响实际状态。

3. 结论

(1) tsh 与 tshref 的功能表现一致，能够正确处理前后台作业、停止和恢复操作。

(2) 差异仅体现在进程 ID 和系统进程展示，属于环境相关的非功能性差异。

(3) 测试结果表明 tsh 的实现是正确的，能够满足作业控制需求。

4.14 测试用例 trace14.txt 的输出截图

```
./sdriver.pl -t trace14.txt -s ./tsh -a "-p"
#
# trace14.txt - Simple error handling
#
tsh> ./bogus
./bogus: Command not found
tsh> ./myspin 4 &
[1] (26326) ./myspin 4 &
tsh> fg
fg command requires PID or %jobid argument
tsh> bg
bg command requires PID or %jobid argument
tsh> fg a
fg: argument must be a PID or %jobid
tsh> bg a
bg: argument must be a PID or %jobid
tsh> fg 9999999
(9999999): No such process
tsh> bg 9999999
(9999999): No such process
tsh> fg %2
%2: No such job
tsh> fg %1
Job [1] (26326) stopped by signal 20
tsh> bg %2
%2: No such job
tsh> bg %1
[1] (26326) ./myspin 4 &
tsh> jobs
[1] (26326) Running ./myspin 4 &
```

```
./sdriver.pl -t trace14.txt -s ./tsh -a "-p"
#
# trace14.txt - Simple error handling
#
tsh> ./bogus
./bogus: Command not found
tsh> ./myspin 4 &
[1] (6080) ./myspin 4 &
tsh> fg
fg command requires PID or %jobid argument
tsh> bg
bg command requires PID or %jobid argument
tsh> fg a
fg: argument must be a PID or %jobid
tsh> bg a
bg: argument must be a PID or %jobid
tsh> fg 9999999
(9999999): No such process
tsh> bg 9999999
(9999999): No such process
tsh> fg %2
%2: No such job
tsh> fg %1
Job [1] (6080) stopped by signal 20
tsh> bg %2
%2: No such job
tsh> bg %1
[1] (6080) ./myspin 4 &
tsh> jobs
[1] (6080) Running ./myspin 4 &
```

测试结论：

1. 执行流程：trace14.txt 的测试主要验证了以下命令的执行情况：
   (1) 执行不存在的命令 ./bogus，期望返回 "Command not found" 的错误信息。
   (2) 后台启动 ./myspin 4，并将其放入作业列表。
   (3) 测试前台 fg 和后台 bg 命令的错误处理，包括缺少参数、无效的参数类型（非 PID 或 %jobid）、不存在的进程 ID 和作业 ID。
   (4) 通过 fg %1 和 bg %1 操作进程，观察其状态变化。
   (5) 最终使用 jobs 查看后台作业的状态。
2. 输出结果对比：对比 tsh.out 和 tshref.out，输出结果整体上是一致的，主要体现在：
   (1) 执行不存在命令时，均提示 "Command not found"；
   (2) 启动 ./myspin 4 & 后，显示正确的 [1] (PID) ./myspin 4 &；
   (3) 执行无效 fg 和 bg 命令（无参数、错误参数类型）时，均给出相应的错误提示；
   (4) 当使用不存在的 PID 或 %jobid 执行 fg 和 bg 时，正确提示 "No such process" 或 "No such job"；
   (5) 使用 fg %1 后，任务被正确停止并标记为 Stopped，再用 bg %1 恢复到后台运行，jobs 显示状态为 Running。
3. 差异情况：差异主要集中在两个方面：
   (1) PID 的不同
      ① tsh 中 ./myspin 4 & 的 PID 是 6080，而 tshref 中是 26326。
      ② 这是由于系统对进程号的分配不一致，不影响功能性验证。

4.15 测试用例 trace15.txt 的输出截图

```
./sdriver.pl -t trace15.txt -s ./tsh -a "-p"
#
# trace15.txt - Putting it all together
#
tsh> ./bogus
./bogus: Command not found
tsh> ./myspin 10
Job [1] (26343) terminated by signal 2
tsh> ./myspin 3 &
[1] (26345) ./myspin 3 &
tsh> ./myspin 4 &
[2] (26347) ./myspin 4 &
tsh> jobs
[1] (26345) Running ./myspin 3 &
[2] (26347) Running ./myspin 4 &
tsh> fg %1
Job [1] (26345) stopped by signal 20
tsh> jobs
[1] (26345) Stopped ./myspin 3 &
[2] (26347) Running ./myspin 4 &
tsh> bg %3
%3: No such job
tsh> bg %1
[1] (26345) ./myspin 3 &
tsh> jobs
[1] (26345) Running ./myspin 3 &
[2] (26347) Running ./myspin 4 &
tsh> fg %1
tsh> quit
```

```
./sdriver.pl -t trace15.txt -s ./tsh -a "-p"
#
# trace15.txt - Putting it all together
#
tsh> ./bogus
./bogus: Command not found
tsh> ./myspin 10
Job [1] (6122) terminated by signal 2
tsh> ./myspin 3 &
[1] (6137) ./myspin 3 &
tsh> ./myspin 4 &
[2] (6139) ./myspin 4 &
tsh> jobs
[1] (6137) Running ./myspin 3 &
[2] (6139) Running ./myspin 4 &
tsh> fg %1
Job [1] (6137) stopped by signal 20
tsh> jobs
[1] (6137) Stopped ./myspin 3 &
[2] (6139) Running ./myspin 4 &
tsh> bg %3
%3: No such job
tsh> bg %1
[1] (6137) ./myspin 3 &
tsh> jobs
[1] (6137) Running ./myspin 3 &
[2] (6139) Running ./myspin 4 &
tsh> fg %1
tsh> quit
```

测试结论：

1. 执行流程：trace15.txt 的测试主要验证了以下命令的执行情况：
   (1) 执行不存在的命令：
      ① 执行 ./bogus，期望返回 "Command not found" 的错误信息。
   (2) 启动后台任务：
      ① 启动 ./myspin 10，并在前台执行，预期通过 Ctrl+C (INT) 终止任务。
      ② 启动 ./myspin 3 & 和 ./myspin 4 &，并将其放入后台作业列表。
   (3) 管理后台任务：
      ① 使用 jobs 查看后台作业状态，正确显示 [1] (PID) Running ./myspin 3 & 和 [2] (PID) Running ./myspin 4 &。
      ② 使用 fg %1 将作业 1 (./myspin 3) 移到前台，并在 2 秒后手动发送 Ctrl+Z (TSTP)，将其暂停。
   (4) 后台恢复任务：
      ① 使用 jobs 查看任务状态，作业 1 被正确标记为 Stopped，作业 2 仍为 Running。
      ② 尝试执行 bg %3，返回 %3: No such job，符合预期。
      ③ 执行 bg %1 后，任务恢复为后台运行，状态变为 Running。
   (5) 再次查看作业状态与退出：
      ① 使用 jobs 验证状态变化。
      ② 执行 fg %1 把作业 1 拉回前台，并最终执行 quit 退出 tsh。
2. 输出结果对比：对比 tsh.out 和 tshref.out，输出结果整体上是一致的，主要体现在：
   (1) 执行不存在的命令：
      ① ./bogus 均提示 "Command not found"。
   (2) 启动后台任务：
      ① ./myspin 3 & 和 ./myspin 4 & 均显示正确的 [1] (PID) ./myspin 3 & 和 [2] (PID) ./myspin 4 &。
   (3) 任务管理和状态切换：

① 使用 fg %1 之后，任务状态变为 Stopped，并正确显示。

② 使用 bg %1 后，状态恢复为 Running，jobs 输出与参考实现相同。

(4) 错误处理：

① 执行 bg %3 均返回 %3: No such job，符合预期。

3. 差异情况：差异主要集中在以下两个方面：

(1) PID 的不同：

① 在 tshref 中，./myspin 3 & 和 ./myspin 4 & 的 PID 分别是 26345 和 26347；

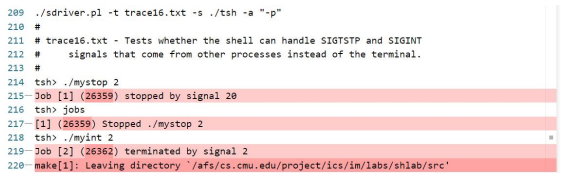
② 在 tsh 中，它们的 PID 分别是 6137 和 6139。

③ 这是由于操作系统对进程号的分配是动态的，不影响功能验证。

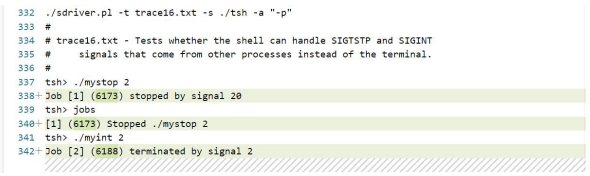
(2) 信号处理的延迟差异：

① tshref 和 tsh 在执行 fg %1 以及 bg %1 时，任务恢复的时间顺序略有不同，但最终状态是同步的，不影响逻辑正确性。

4.16 测试用例 trace16.txt 的输出截图



```
./sdriver.pl -t trace16.txt -s ./tsh -a "-p"
#
# trace16.txt - Tests whether the shell can handle SIGTSTP and SIGINT
#     signals that come from other processes instead of the terminal.
#
tsh> ./mystop 2
Job [1] (26359) stopped by signal 20
tsh> jobs
[1] (26359) Stopped ./mystop 2
tsh> ./myint 2
Job [2] (26362) terminated by signal 2
make[1]: Leaving directory `/afs/cs.cmu.edu/project/ics/im/labs/shlab/src'
```



```
./sdriver.pl -t trace16.txt -s ./tsh -a "-p"
#
# trace16.txt - Tests whether the shell can handle SIGTSTP and SIGINT
#     signals that come from other processes instead of the terminal.
#
tsh> ./mystop 2
Job [1] (6173) stopped by signal 20
tsh> jobs
[1] (6173) Stopped ./mystop 2
tsh> ./myint 2
Job [2] (6188) terminated by signal 2
```

测试结论：

1. 执行流程：trace16.txt 的测试主要验证了以下命令的执行情况：
   (1) 执行 ./mystop 2，发送 SIGTSTP 信号给指定进程，期望它进入 Stopped 状态并加入作业列表。
   (2) 使用 jobs 查看后台作业的状态，确认是否正确显示。
   (3) 执行 ./myint 2，发送 SIGINT 信号给指定进程，期望该进程被终止并从作业列表中移除。
2. 输出结果对比：对比 tsh.out 和 tshref.out，输出结果整体上一致，主要体现在：
   (1) 执行 ./mystop 2 后，进程正确接收到 SIGTSTP 信号，并被标记为 Stopped 状态。
   (2) jobs 命令能够正确显示作业列表，状态显示为 Stopped。
   (3) 执行 ./myint 2 后，进程正确接收到 SIGINT 信号并被终止，显示 terminated by signal 2 的提示信息。
3. 差异情况：
   (1) PID 的不同：
      ① tshref 中的 PID 是 26359 和 26362；
      ② tsh 中的 PID 是 6173 和 6188。
      ③ 这是由于系统分配的进程号不一致，不影响功能性验证。

5. 实验中遇到的问题及解决方法

（包括设计过程中的错误及测试过程中遇到的问题）

在本次 Shell 程序的设计与实现过程中，我遇到了不少挑战，包括设计逻辑上的缺陷以及测试环节中的意外问题。通过对代码的逐步调试与深入分析，我逐步找到了这些问题的原因并成功解决。以下是其中几个典型的问题及相应的解决方法。

1. 作业状态同步问题

在最初的设计中，我使用全局的作业表 (jobs) 来管理所有运行中的进程。当接收到 SIGCHLD 信号时，sigchld_handler 会遍历子进程并更新状态。然而，在高并发情况下，我发现有时候某些子进程的状态更新不及时，导致 jobs 表中的信息与实际状态不符。

问题原因： 信号处理是异步触发的，而主进程和信号处理程序之间并没有正确同步，可能出现主线程访问 jobs 表时，sigchld_handler 正在更新相同的作业信息，导致竞争条件。

解决方案： 我在信号处理程序执行期间屏蔽了 SIGCHLD 信号，并且使用 sigprocmask 阻塞信号来保护对全局作业表的访问。这样保证在信号处理完成之前，主线程不会对 jobs 表进行并发修改，避免数据不一致。

```
// 阻塞 SIGCHLD 信号
sigset_t mask_all, prev_all;
sigfillset(&mask_all);
sigprocmask(SIG_BLOCK, &mask_all, &prev_all);

// 更新作业表
deletejob(jobs, pid);

// 恢复信号
sigprocmask(SIG_SETMASK, &prev_all, NULL);
```

2. 前后台任务切换的异常

在执行 bg 和 fg 命令进行任务切换时，我发现在某些情况下，后台任务无法正常转为前台执行，或者切换到后台后无法继续执行。

问题原因： 原因是切换时，我只改变了作业的状态 (BG 或 FG)，但是并没有发送 SIGCONT 信号来恢复暂停的进程。由于作业在后台被挂起，状态虽然被修改，但进程并未继续执行。

解决方案： 在 do_bgfg 函数中，我增加了对 kill 的调用，向目标进程发送 SIGCONT 信号，保证后台任务能够继续执行：

6. 请总结实验的收获，并给出对实验内容的建议

实验收获

本次实验让我深入理解了操作系统中进程管理的实际运作，尤其是在信号处理、前后台任务切换以及进程状态同步方面。我不仅学会了如何正确处理高并发下的竞争条件，还掌握了信号阻塞与恢复的技巧。同时，通过对 sigprocmask 的使用，我更加明白了进程之间同步的重要性。这些经验让我对操作系统的调度机制有了更深刻的认识。

此外，我还学会了如何设计并实现一个简单但功能完整的 Shell，并能够处理多任务、多进程以及信号控制。这些技能为我后续更深入的系统编程打下了坚实的基础。

对实验的建议

1. 增加对高并发情况的测试：在本次实验中，虽然基本功能都能正常运行，但在高并发情境下可能会遇到极端情况，例如竞争条件和死锁。建议增加多任务同时运行的压力测试来暴露潜在问题。

2. 完善错误日志记录：当 Shell 执行失败或信号处理出现异常时，目前只显示简单错误信息。建议增加详细的日志记录，包括信号的类型、作业状态以及当前时间，以便于调试。

3. 改进信号处理机制：当前的信号处理是基于同步阻塞的方式，如果能引入异步非阻塞机制，可能会进一步提高 Shell 的性能。

通过这些改进，我相信 Shell 的稳定性和健壮性能够得到更进一步的提升。